UNIVERSITA' DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Master's Degree in Artificial Intelligence and Data Engineering

Intelligent Monitoring of Escape Routes via Computer Vision

Advisors:

Eng. Giustino Claudio Miglionico

Prof. Francesco Marcelloni

Prof. Pietro Ducange

Author:

Andrea Innocenti

ACADEMIC YEAR 2025/2026

Abstract

[Insert thesis abstract in English here. This should provide a concise summary of the research problem, methodology, key results, and conclusions - typically 150-300 words.]

Contents

List of Figures

Chapter 1

Introduction

1.1 Motivation

[Contextualize the problem and justify the work]

1.2 Research Question

[Clear statement of the research question]

1.3 Thesis Outline

[Overview of remaining chapters]

Chapter 2

State of the Art

2.1 Vision-Based Safety Monitoring in Built Environments

Safety inspections of buildings and workplaces have traditionally been carried out by human operators who periodically walk the premises, identify hazards, and verify compliance with safety regulations. The literature consistently identifies the same limitations of this manual process: it is costly, it scales poorly with the size of the monitored facility, and its low frequency leaves long blind intervals during which a hazard can persist undetected [10, 5]. Computer vision offers a natural path to automation, since most safety-relevant conditions - a blocked corridor, a missing helmet, a misplaced object - are directly observable in images, and camera infrastructure is comparatively cheap to deploy and maintain.

The condition of evacuation routes is a recurring subject in this literature. Ercan and Mathyalakan [10] address clutter in the common areas of Singapore public housing, where residents' belongings - bicycles, laundry racks, shoe racks, plants - accumulate along corridors and may hinder emergency response; safety guidelines mandate a clear escape passage, but enforcement through on-site inspection is infrequent and labour-intensive. Their system detects clutter with YOLO-family object detectors mounted on an autonomous mobile robot and reports violations, with location and type of obstruction, to a remote database. Along the same line, fire-safety regulations are the explicit target of Chen et al. [5], who observe that automated compliance checking has historically been confined to the *design* phase, operating on structured building models, and therefore cannot capture hazards that arise only during building *operation* - obstructed exits, relocated equipment, informal modifications of the space. Their proposed remedy combines deep-learning perception modules with a knowledge graph encoding spatial relationships and regulatory rules, so that raw on-site imagery can be turned into structured, regulation-aware compliance reports.

A third strand concerns construction sites, arguably the most mature application domain for vision-based safety monitoring. Beyond detecting unsafe configurations

of equipment and personnel, recent work has concentrated on the *reliability* of the monitoring signal itself: Zaidi et al. [38] note that systems classifying each video frame in isolation inevitably produce false alarms whenever a single frame is misdetected, and introduce a temporal-analysis module that aggregates evidence over a window of frames before raising an alarm. The concern they articulate - that a monitoring system crying wolf is quickly ignored by its operators - is general, and returns in Section 2.5 as a central design constraint for the present work.

Across these systems three common traits emerge. First, the sensing infrastructure consists of fixed or slowly-moving cameras observing a known scene, which makes the scene layout itself prior knowledge that the algorithms can exploit. Second, the monitoring must run continuously and in (near) real-time, which constrains the admissible computational budget. Third, imagery of building interiors is privacy-sensitive, and streaming it to a remote server is often undesirable or prohibited; together with bandwidth and per-camera cost considerations, this pushes the computation toward the network edge, a constraint examined in the next section.

2.2 Object Detection on Edge Devices

The most direct way to recognise an obstruction in an image is to detect the obstructing object explicitly. Deep-learning object detectors are commonly divided into two-stage models, which first propose candidate regions and then classify them, and one-stage models, which predict classes and bounding boxes in a single forward pass [22]. The YOLO family [26] is the prototypical one-stage design and has become the de-facto standard for real-time applications: it processes the whole image with one convolutional network, trading a modest amount of accuracy for a substantial gain in speed over two-stage models. Successive YOLO generations, together with dedicated lightweight architectures, quantisation, and inference engines such as TensorRT and OpenVINO, have progressively lowered the computational cost of detection to the point where it fits the resource envelope of single-board computers; Mittal [22] surveys this landscape and the reasons driving it - bandwidth reduction, latency, energy, and the privacy advantage of keeping image data on the device that captured it.

How far this paradigm can be pushed on low-cost hardware is well documented by the line of work on personal protective equipment (PPE) monitoring, a safety application whose deployment constraints closely mirror those of exit monitoring. Gallo et al. [12] argue against the prevailing cloud-based architectures - which require sustained bandwidth, a reliable connection, and the off-site transfer of workers' images - and deploy PPE detection entirely on a Raspberry Pi 4 with a USB neural accelerator, selecting YOLOv4-Tiny as the best accuracy/throughput compromise at about 6.8 frames per second. On the same dataset, Miglionico et al. [21] evaluate the YOLOv5 size spectrum on identical hardware, with the nano variant reaching 7.9 frames per second at 81.7% mAP@50, and later extend the comparison to YOLOv11 and the transformer-based RT-DETR on an NVIDIA Jetson Nano [20]: the nano

model with TensorRT optimisation achieves 10.6 frames per second within a 3 W power budget, whereas transformer detectors, although competitive in accuracy, prove too slow for real-time edge inference. Taken together, these studies establish that modern compact detectors deliver usable accuracy at interactive frame rates on hardware costing tens to a few hundred euros - a premise this thesis builds upon.

Closer to the problem addressed here, object detectors have been applied to the monitoring of evacuation routes themselves. Besides the corridor-clutter robot of Ercan and Mathyalakan [10] discussed above, Shin et al. [32] use a YOLOv8 model on frames from pre-existing CCTV cameras to detect loads and obstacles in the corridors of complex buildings, and convert the detections, through a homography-based perspective correction, into an estimate of the *effective evacuation width* that remains available; after augmenting and re-labelling a custom dataset they report near-saturated metrics (precision 0.99, recall 1.0) in their static test setting. This confirms the feasibility of detector-based egress monitoring on existing camera infrastructure, the same deployment model assumed in this thesis.

The limits of the paradigm, however, are structural rather than incidental. First, a detector can only find what it was trained to find: the obstacle vocabulary is fixed at training time, while the space of objects that can block an exit is open-ended, so any closed-set detector leaves coverage gaps by construction. Second, the supervised regime requires a labelled dataset whose collection and annotation dominate the deployment cost, and whose biases bound the model's generalisation - Shin et al. [32] themselves must re-label and augment their data to cope with classes absent from public datasets. Third, detection accuracy degrades when objects are partially occluded or truncated, a pervasive condition in cluttered corridors; an entire research thread surveyed by Ouardirhi et al. [23] is devoted to occlusion-handling strategies, and its very existence signals that the problem is far from solved. Within this thesis, the pure object detection based pipeline (P1) inherits both the strengths and these limitations; the approaches reviewed in the next two sections trade the ability to *name* the obstacle for independence from a predefined obstacle vocabulary.

2.3 Reference-Based Change Detection

When the camera is fixed, an alternative to recognising obstacles is to detect that the scene has *changed* with respect to a stored reference of its clear state. This idea descends from classical background subtraction, where a statistical background model is maintained over time and pixels deviating from it are labelled foreground. As Santana et al. [30] point out, however, conventional background models adapt continuously and eventually absorb long-standing foreground objects into the background - precisely the wrong behaviour for obstruction monitoring, where a cart abandoned in front of an exit must keep raising an alarm for as long as it stays there. Route obstruction therefore calls for comparison against a *fixed* reference of the known-clear scene rather than against an adaptive background estimate.

A learned alternative to this fixed pixel-level reference is to model the scene's

normal appearance with a generative network - a variational autoencoder (VAE) or a generative adversarial network (GAN) - and to flag whatever it cannot reproduce as change. The route is attractive in principle but carries structural liabilities that weigh against it here. Bouman and Heskes [2] show that VAE-based detectors are fundamentally unreliable: a sufficiently expressive decoder reconstructs even out-of-distribution inputs faithfully, suppressing the very reconstruction error the method relies upon. GAN-based approaches are prone to mode collapse - the generator fails to cover the full range of normal appearances, and the uncovered modes are then mistaken for anomalies - and their adversarial training is notoriously sensitive to architecture and hyperparameter choices, making stable convergence hard to guarantee [14, 18]. Both families further impose a training cost markedly higher than a direct comparison against a stored reference, compounding their incompatibility with edge deployment.

Sidestepping this generative bottleneck, two recent works compare the current frame against the stored reference directly - in geometry or in a learned feature space - and apply the paradigm to evacuation routes. Talebi et al. [33] propose a door-centric formulation of exit-blockage detection: instead of enumerating possible obstacles, the system monitors only the visual state of the door itself. A lightweight YOLOv11 network is trained on a single class, the door, and the bounding box it predicts at run time is compared - in geometry and overlap - against the box recorded in the unobstructed reference state; a blockage is inferred whenever the visible portion of the door deviates persistently from the reference. The authors argue that this bypasses the structural weakness of obstacle catalogues identified in Section 2.2, since any object large enough to obstruct the doorway alters the door's visible geometry regardless of its category, and they demonstrate real-time operation on a Raspberry Pi. The trade-off is the coarseness of the signal: the method observes a single bounding box, so it cannot localise the obstruction, and changes that do not alter the door's visible extent - or doors that the detector fails to find under unusual lighting - fall outside its reach. The door-centric pipeline (P2) developed in this thesis follows this design.

Santana et al. [30] push the comparison to the pixel level with a Siamese architecture. Two weight-sharing U-Net encoders process the reference frame and the current frame, their representations are compared at multiple levels of abstraction, and a decoder produces a binary segmentation mask of the changed regions; training on the generic change-detection corpus CDNet2014 teaches the network what constitutes a meaningful change, without committing to any object vocabulary. Applied to PetrobrasROUTES, a dataset of obstructed and abandoned objects on escape routes of industrial plants, the approach proves markedly more robust to noise and illumination variation than classical background-modelling baselines, because the comparison happens in a learned feature space rather than in pixel intensities. This is the design the Siamese change-detection pipeline (P4) of this thesis builds upon.

Reference-based methods occupy an attractive middle ground for the problem at hand: they are object-agnostic, produce a spatial localisation of the change (in the

Siamese variant), and exploit the fixed-camera deployment model naturally. Their residual requirements are nonetheless non-trivial: the Siamese comparator must still be *trained* to distinguish meaningful change from nuisance variation, which presupposes a labelled change-detection corpus, and every method in this family remains exposed to reference staleness - the stored clear-state image must be re-acquired whenever the normal appearance of the scene legitimately changes. The family of methods reviewed next pushes label-independence one step further, dispensing with change supervision altogether.

2.4 Unsupervised Anomaly Detection

The third family of approaches abandons supervision entirely: the system learns a model of *normality* from unlabelled normal data and flags, at test time, whatever deviates from it. This one-class formulation matches the data asymmetry of safety monitoring - normal observations are abundant and free, anomalous ones are rare, heterogeneous, and expensive to stage - and it is the formulation adopted by the third pipeline of this thesis.

In video surveillance, anomaly detection is a mature research area, surveyed among others by Duong et al. [9]. The dominant problem setting is the detection of abnormal *human* behaviour - fighting, intrusions, abandoned luggage - with models trained on normal footage and evaluated on benchmarks such as UCSD Ped, CUHK Avenue, and ShanghaiTech. Two observations from this literature carry over directly to exit monitoring. First, anomaly is *context-dependent*: an event normal in one scene may be anomalous in another [9, 24], which implies that normality is best modelled per camera rather than globally - the design choice at the heart of the per-camera memory of Chapter 3. Second, the recurrent failure sources reported across methods - occlusion, low illumination, dynamic backgrounds and sensor noise [9] - are environmental rather than algorithmic, and any deployable system must budget for them explicitly. Patrikar and Parate [24] review the same field from the deployment side, observing that anomaly detection is a delay-sensitive application for which round-trips to a cloud back-end are ill-suited, and that the natural convergence point is anomaly detection executed on edge devices co-located with the cameras - the architecture assumed throughout this thesis.

A parallel and, for the present work, more directly relevant line of research developed in *industrial* visual inspection, where products photographed under a fixed viewpoint must be screened for defects never seen during training. The release of the MVTec AD benchmark [1] crystallised the modern formulation: only defect-free images are available for training, and methods are judged on both image-level detection and pixel-level localisation. The best-performing family of methods sidesteps representation learning altogether, describing normality directly in the feature space of a network pre-trained on natural images: SPADE [7] retrieves nearest neighbours of multi-scale features from a gallery of normal images, and PaDiM [8] fits a per-position Gaussian to the features of aligned normal samples. PatchCore [28] re-

fined this embedding-based approach into the de-facto standard: locally aggregated patch features from a frozen backbone are stored in a memory bank, compressed by greedy coreset subsampling to a small fraction of its original size with negligible performance loss, and queried at test time by a re-weighted nearest-neighbour rule that yields both a scalar anomaly score and a spatial anomaly map. On MVTec AD, PatchCore achieves an image-level AUROC above 99%, but the reason to prefer it here emerges from how it compares with its two predecessors, SPADE and PaDiM. PaDiM is parametric: it estimates a per-position Gaussian in feature space, which becomes reliable only with a sizable sample of normal frames - a requirement at odds with a memory built from a single reference view. SPADE belongs instead to the same nonparametric, position-agnostic family as PatchCore, but predates its refinements: it scores each location by plain nearest-neighbour retrieval against an uncompressed gallery, whereas PatchCore adds local neighbourhood aggregation, a re-weighted scoring rule, and coreset subsampling that jointly raise detection accuracy while bounding the memory. The net effect is a method that memorises exemplar patches rather than fitting a distribution, matches them regardless of their position, and stays training-free and computationally frugal at inference. It is this combination that makes a one-shot, per-camera memory viable on constrained hardware, and it is the reason PatchCore, in adapted form, is adopted as the core of pipeline P3 (Chapter 3). The position-agnostic nature of this matching has a further consequence exploited in Section 2.5: nuisance patterns deliberately inserted into the memory, such as cast shadows and light beams, are then recognised as normal wherever they appear in the frame, whereas a per-position model such as PaDiM would have to observe each nuisance at every location to absorb it.

The transfer from industrial inspection to evacuation-route monitoring is conceptually natural: a fixed camera observing a doorway plays the role of the fixed inspection viewpoint, the clear exit is the defect-free product, and an obstruction is a structural defect of arbitrary, unforeseen appearance. The correspondence is not perfect, however. Industrial imagery is acquired under controlled, repeatable illumination, whereas a corridor is subject to daylight cycles, cast shadows, and reflections; the nuisance variability that industrial benchmarks deliberately suppress becomes, in this domain, the principal source of false alarms. Techniques for managing precisely this gap are the subject of the next section.

2.5 Robustness to Illumination Nuisances and False Alarms

For a monitoring system that runs unattended, the false-positive rate is not a secondary metric but the property that decides whether the system is usable at all: operators quickly learn to ignore an alarm that fires on harmless events, at which point the system provides negative value. The literature attacks the problem from two complementary directions - making the *decision* more conservative, and making

the *model* more robust to the nuisance factors that trigger spurious detections.

The first direction is exemplified by Zaidi et al. [38], already encountered in Section 2.1: since per-frame classification amplifies every transient misdetection into an alarm, they accumulate evidence over a temporal window and raise an alarm only when the fraction of hazardous frames exceeds a threshold, measurably reducing false alarms during live monitoring. Temporal aggregation of this kind is largely orthogonal to the underlying detector and could be layered onto any of the pipelines considered in this thesis; it does not, however, help against nuisance factors that are *persistent* rather than transient - a cast shadow that sits on the floor for hours defeats any voting window.

Persistent nuisances dominate in the fixed-camera one-class setting, and illumination is by far the most important among them. Cao et al. [3] provide the systematic evidence: their M^2AD benchmark images about a thousand objects under 120 combinations of viewpoint and illumination and shows that state-of-the-art visual anomaly detection methods, PatchCore included, degrade severely when test-time illumination deviates from the training condition.

Among illumination effects, cast shadows are the most structured and the most deceptive, since they introduce sharp, object-like contours that low-level features cannot distinguish from physical objects. The classical remedy is to detect and remove shadows before analysis, a research line with a long history in traffic and outdoor surveillance [29]; but removal adds a processing stage, and its own failures propagate downstream. An alternative, recently explored by Ju and Reibman [16] for camera-based action recognition, is to make the model itself shadow-tolerant by *injecting synthetic shadows into the training data*. Their study offers two findings of direct relevance here. First, shadow augmentation applied to shadow-free training data substantially improves accuracy on out-of-distribution test sets containing real, heavy shadows - the exact configuration of a one-shot memory built from a clean reference image and tested on frames with environmental shadows. Second, generic photometric augmentation is not a substitute: globally darkening the training images *worsens* performance, confirming that robustness to localised, structured darkening requires explicitly structured augmentation rather than wider brightness ranges. The shadow and light augmentation scheme adopted in pipeline P3 (Chapter 3) follows this principle.

2.6 Positioning of This Work

The deployment scenario of this thesis - a fixed camera per emergency exit, processing on a low-cost edge device, alarms for arbitrary obstructions - induces four requirements against which the reviewed families can be measured:

1. **Open-set coverage**: any object blocking the route must be detectable, including categories never seen before;

2. **Label independence**: deploying on a new camera should require no labelled examples of obstructions;

3. **Edge compatibility**: inference must run continuously on resource-constrained hardware;

4. **Nuisance robustness**: illumination changes, shadows, and activity behind glass doors must not raise alarms. Pinpointing *where* the obstacle lies is not an end in itself - a blocked doorway is self-evident to an operator - but a spatially-resolved response serves this requirement, since it lets known nuisance regions be filtered out by location.

Table 2.1 summarises how the four approach families reviewed in this chapter - each instantiated by one of the pipelines compared in this thesis - relate to these requirements.

Approach	**Training data**	**Obstacle coverage**	**Localisation**	**Per-camera deployment**
Object detection (P1) [12, 20, 32]	labelled boxes for each obstacle class	closed-set	bounding boxes	none (shared model)
Door-centric monitoring (P2) [33]	door-class labels only	open-set (indirect)	door box only	reference state of the door
Siamese change detection (P4) [30]	generic change-segmentation corpus	open-set	pixel mask	reference frame
One-class anomaly detection (P3) [28]	none (normal images only)	open-set	anomaly map	single reference image

Table 2.1: Approach families for exit-obstruction detection, each instantiated by one of the four pipelines of this thesis, against the requirements of the target deployment.

No single family dominates. Supervised detection offers semantic interpretability - it names the obstacle - and benefits from a mature edge-deployment toolchain [22, 20], but its closed vocabulary and labelling cost conflict with requirements 1 and 2. Door-centric monitoring [33] is open-set and computationally frugal; its detection signal is coarse and frame-level, and a strong shadow or reflection that occludes part of the door surface can shrink the detected bounding box into a spurious blockage alert - a milder but real exposure to requirement 4. Siamese change detection [30] is open-set and pixel-precise, yet still depends on a labelled change corpus and on the maintenance of an up-to-date reference frame. One-class anomaly detection in the

PatchCore style [28] covers requirements 1 to 3 in a single design - open-set, label-free, and frugal enough for edge deployment - but concentrates its vulnerability on requirement 4: embedding-based methods presume photometric consistency between reference and test conditions [3], the very assumption that daylight, shadows, and reflections violate in an uncontrolled corridor. Its most complete profile on paper thus comes paired with the sharpest exposure to the setting's dominant nuisance.

What this review exposes is not a settled choice but an open question. No prior work has weighed the four families against one another on a common evacuation-route setting under common edge constraints, so which of them is genuinely most effective for exit-obstruction detection - and at what cost in accuracy, false alarms, and inference budget - has never been measured. Establishing this is the primary objective of the thesis: object detection (P1), door-centric monitoring (P2), one-class anomaly detection (P3), and Siamese change detection (P4) are each implemented as a pipeline, adapted to the same fixed-camera, per-exit deployment, and compared on equal terms - on a shared dataset, under a single experimental protocol, and against the four requirements set out above (Chapter 5).

The ambition, however, reaches beyond this single application. Two of the pipelines lean on locating the exit - the door-centric baseline by construction, and the anomaly-detection pipeline through the door region and reference it monitors - so they are not turnkey detectors that drop unchanged into an arbitrary scene. What generalises is the concept behind them: treating obstruction detection as the per-camera monitoring of a known-clear region of interest for arbitrary, unforeseen obstacles, and choosing among competing strategy families through a shared dataset and protocol. Recast around a different region of interest, the same approach carries over to other fixed-camera safety-monitoring settings - industrial walkways, restricted-access zones, other critical passages.

Chapter 3

Method

3.1 Objective

The goal of this work is to detect, on an edge device, when an obstacle blocks an evacuation route (a door or a corridor). Among the pipelines compared in this thesis, this chapter develops two complementary ones. *Unsupervised Anomaly Detection with PatchCore* (P3) learns only from normal, unobstructed scenes and requires no labelled examples of obstructions. *Single-Class Obstruction Detection with YOLO* (P1) takes the opposite, supervised route: a deep object detector is fine-tuned to localise obstacles explicitly, trading annotation effort for the ability to state *where* the obstruction is and to ignore scene changes that are not objects. The following sections derive the components of each method, their mathematical formulation, and the data on which they operate; implementation choices and numerical settings are deferred to Chapter 4, and the empirical evaluation to Chapter 5.

3.2 Pipeline Overview

3.2.1 Unsupervised Anomaly Detection with PatchCore (P3)

Motivation and design rationale

The detection of emergency-exit occlusions presents a fundamental labelling bottleneck. Normal scenes - clear, unobstructed doorways and corridors - are easy to capture at scale, but the space of possible obstructions is open-ended: wheelchairs, carts, stretchers, waste bins, cardboard boxes, and any combination thereof can occupy a doorway in arbitrary configurations, orientations, and lighting conditions. Collecting and labelling a representative sample of anomalous scenes is time-consuming, and any finite labelled set risks leaving coverage gaps on obstacle types not seen during training.

P3 sidesteps this bottleneck by framing occlusion detection as *one-class anomaly detection*: the system learns only from normal scenes and, at inference time, flags any image whose feature representation deviates significantly from what was memorised

as normal. This removes the need for labelled anomalous examples entirely. The choice to adapt PatchCore [28], originally proposed for industrial surface inspection, to the emergency-exit domain is motivated by three properties that align with the deployment context:

1. **One-shot operation**: a single reference photograph of each monitored location is sufficient to build the entire anomaly model. In a real deployment, each fixed camera captures one reference frame of the clear exit; no further data collection is required.

2. **Spatial resolution**: PatchCore operates at the patch level rather than producing a single image-level embedding, preserving the spatial structure of the anomaly and enabling heatmap visualisation and spatially-aware filtering.

3. **Edge compatibility**: inference - feature extraction through a frozen backbone followed by a nearest-neighbour lookup in a compact memory - runs entirely on CPU, requires no gradient computation, and can be parallelised per camera without shared state.

Among the four pipelines, P3 is the most data-efficient: it needs only a single normal reference image and a pre-trained backbone, with no annotation effort to deploy on a new camera. The trade-off is that it cannot identify *which* obstacle is present - it only signals that something is anomalous - and it is, in principle, sensitive to any scene change, including non-obstacle changes such as lighting variations, shadows, or background movement. Managing these false-positive sources is the central methodological challenge of this chapter.

Key intuition

The core intuition is that the intermediate representations of a deep convolutional network pre-trained on a large natural-image corpus are broadly transferable: they capture generic visual patterns (textures, edges, object parts) that remain discriminative across domains, including surveillance footage of building interiors. A patch belonging to a normal scene of a fire-exit door yields a feature vector close to other normal patches in this embedding space, whereas a patch belonging to an unexpected obstacle - the wheel of a cart, the fabric of a stretcher, the surface of a box - yields a feature vector distant from all stored normal representations, hence a high anomaly score.

The memory is built from augmented variants of a single reference image rather than from a large dataset, exploiting the fixed-camera assumption: the camera does not move, so the background is stable across days and the only source of variation is illumination. Photometric augmentation at build time teaches the memory to tolerate this variation, making the decision threshold robust to normal fluctuations without requiring multiple observation days.

It is worth stressing why the alternative - populating the memory with a collection of *similar* exit scenes (many different corridors, many different doors) - would be both infeasible and counterproductive. Such scenes are visually too heterogeneous: a memory mixing dozens of unrelated corridors and doorways would model the generic appearance of "a corridor" or "a door", which is precisely the wrong target. Its feature manifold would be so broad that an obstacle in any specific scene could fall close to some unrelated normal scene in the bank and be scored as normal, destroying the detector's sensitivity. The task is not to recognise corridors and doors in general, but to recognise *this* particular exit as it normally looks. This is the natural consequence of the deployment model, in which each camera monitors exactly one exit route: the normal class for that camera is its own fixed scene, and the only relevant variability is the illumination and the background change of that scene over time. A per-camera memory built from one reference image is therefore not a limitation imposed by scarce data, but the correct modelling choice for the objective. The figure 3.1 shows all the pipeline steps.

Feature extraction

Let I_{ref} denote the normal reference image of a monitored location. Every image is resized to a fixed resolution and passed through a frozen convolutional backbone f. Following the original PatchCore formulation, features are read from two intermediate blocks, ℓ_2 and ℓ_3: the shallower block preserves local detail while the deeper block carries semantic context, and their combination balances the two. The shallowest block encodes mostly low-level texture with little semantic content, and the deepest block is too abstract and biased toward the pre-training classes; both are therefore excluded.

The deeper feature map is upsampled bilinearly to the spatial resolution of the shallower one and concatenated along the channel axis, yielding a joint tensor of patch vectors arranged on a $h \times w$ grid. A local average pooling over a 3×3 neighbourhood (the neighbourhood aggregation of [28]) is then applied: each patch vector is replaced by the mean of its 3×3 spatial neighbourhood,

$$\phi(x) \;=\; \frac{1}{|\mathcal{N}_3(x)|} \sum_{x' \in \mathcal{N}_3(x)} \tilde{\phi}(x'), \tag{3.1}$$

where $\tilde{\phi}(x')$ is the raw concatenated feature at grid location x' and $\mathcal{N}_3(x)$ is the 3×3 window centred at x. This enlarges the receptive field of each vector and reduces sensitivity to pixel-level noise. The result is a set of $N_p = h \cdot w$ patch feature vectors per image.

Build phase

The build phase constructs the *memory bank* $\mathcal{M}$, the finite set of normal patch features against which every test patch will later be compared. It proceeds in three

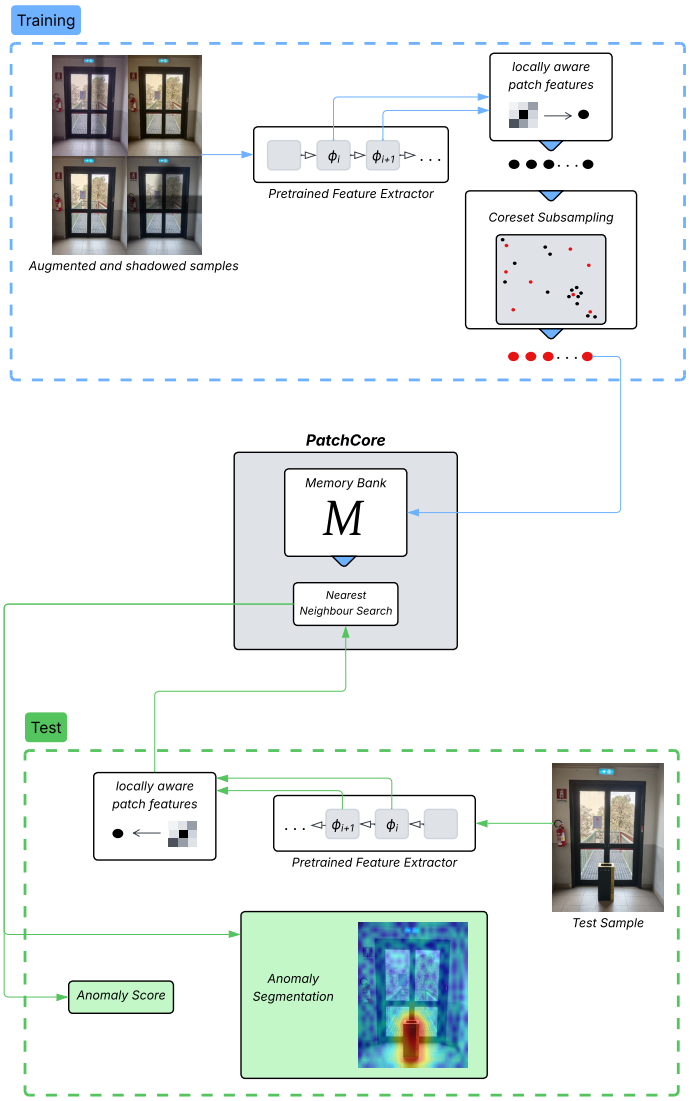


Figure 3.1: Schematic of the P3 pipeline. The memory bank is built from augmented variants of a single reference image, and test patches are scored by their distance to the nearest bank entry. The decision threshold is calibrated on additional augmented variants.

steps - augmentation, feature accumulation, and coreset reduction - each of which is detailed below. The guiding principle is that $\mathcal{M}$ must represent the full appearance of the normal scene under its expected illumination variation, yet remain small enough for a real-time nearest-neighbour search on an edge device.

The build phase is a *one-off, offline* operation, performed once when the camera is installed and its reference frame I_{ref} is captured. The resulting memory bank is then stored and reused unchanged for every subsequent inference; nothing in the build phase runs at monitoring time. A rebuild is required only when the *normal* reference scene itself changes in a way that is not a transient occlusion -

for example a permanent rearrangement of the furniture in view, a repainting, or a change of the camera's mounting - so that the old I_{ref} no longer describes the normal state. Routine illumination changes and background variation over time are deliberately *not* a reason to rebuild: they are exactly what the augmentation and the calibration are designed to absorb. This separation between a rare, offline build and a continuous, lightweight inference is what makes the method practical on an edge device.

Step 1 - Augmentation. A single reference frame is too narrow a description of the normal scene: it captures one instant of illumination, whereas the same fixed camera will, over days, see the scene under brighter or dimmer light, warmer or cooler colour temperature, and varying sensor noise. To let the memory tolerate this variation, n_{aug} augmented variants are generated from I_{ref}, where n_{aug} is the number of bank variants. The augmentations are predominantly photometric - brightness, contrast, saturation, colour-temperature shifts, additive noise, and a mild blur - each motivated by a physical source of normal variation (time of day, light type, sensor noise, slight defocus). *Large* geometric transforms (flips, wide rotations, crops) are deliberately avoided: the camera is physically fixed, so orientation is semantically meaningful and must not be treated as a source of invariance, otherwise the memory would also accept an obstacle appearing in a flipped or arbitrarily relocated position as normal.

A controlled exception is a *small* geometric perturbation, off by default and enabled only for cameras subject to micro-vibration. When active, each variant is additionally shifted by a random translation of at most $\delta_{\max}$ pixels on each axis and rotated by a random angle of at most $\rho_{\max}$ degrees, with $\delta_{\max}$ of a few pixels and $\rho_{\max}$ of a couple of degrees. Its purpose is opposite to that of the forbidden large transforms: rather than making the memory invariant to where an obstacle sits, it makes the memory tolerant to a slight global displacement of the *whole* fixed scene, as produced by mechanical vibration or a small mounting drift. Such a displacement moves every patch by the same tiny amount, so a few-pixel jitter keeps a normal scene normal without ever spanning the distance an obstacle would cover. This is the cheapest of the available robustness measures, since it acts entirely at build time and adds no inference-time cost.

Step 2 - Feature accumulation. Each of the n_{aug} variants is passed through the feature pipeline of equation (3.1), producing N_p patch vectors per variant. Stacking the patches of all variants gives the *raw memory*

$$\mathcal{M}_{\text{raw}} = \bigcup_{v=1}^{n_{\text{aug}}} \left\{ \phi^{(v)}(x) \;:\; x \in \text{grid} \right\}, \qquad |\mathcal{M}_{\text{raw}}| = n_{\text{aug}} \cdot N_p, \tag{3.2}$$

where $\phi^{(v)}(x)$ is the aggregated feature at grid location x of variant v. This mirrors the memory construction of [28], $\mathcal{M} = \bigcup_{x_i \in \mathcal{X}_N} \mathcal{P}(\phi(x_i))$, with the nominal

training set $\mathcal{X}_N$ here instantiated by the augmented variants of the single reference I_{ref}. Each spatial location of the scene thus contributes n_{aug} nearby vectors - one per illumination variant - which together trace out the small region of feature space that a normal patch at that location can occupy. This is exactly the tolerance the detector needs: at test time a normal patch, under any illumination within the augmented range, will fall close to at least one of these vectors and receive a low anomaly score.

Step 3 - Coreset reduction. The raw memory is highly redundant: adjacent grid locations on flat surfaces (a wall, a floor, a door panel) and the n_{aug} illumination copies of each location produce many near-duplicate vectors. Storing all of them would inflate both the memory footprint and the nearest-neighbour cost without improving coverage. The memory is therefore compressed to a coreset $\mathcal{M} \subset \mathcal{M}_{\text{raw}}$ of size a fraction p of the original - the *coreset ratio* - that minimises the worst case distance of any raw vector to its closest retained vector [31, 28]:

$$\mathcal{M}^* = \arg \min_{\mathcal{M} \subset \mathcal{M}_{\text{raw}}} \max_{m \in \mathcal{M}_{\text{raw}}} \min_{m' \in \mathcal{M}} \|m - m'\|_2. \tag{3.3}$$

This minimax (facility-location) objective is NP-hard, so it is approximated by a greedy farthest-point procedure: starting from one random point, the algorithm repeatedly adds the still unselected vector that is *farthest* from the current selection,

$$m_t = \arg \max_{m \in \mathcal{M}_{\text{raw}} \setminus \mathcal{M}_t} \min_{m' \in \mathcal{M}_t} \|m - m'\|_2, \qquad \mathcal{M}_{t+1} = \mathcal{M}_t \cup \{m_t\}. \tag{3.4}$$

Choosing, at each step, the point that maximises its minimum distance to the already-selected set spreads the retained vectors as evenly as possible over the occupied region of feature space: dense clusters of near-duplicates are summarised by a few representatives, while isolated points - which carry unique information about the normal scene - are kept. The procedure stops when $|\mathcal{M}| = \lceil p \cdot |\mathcal{M}_{\text{raw}}| \rceil$, subject to a small floor on the number of retained points so that even a very low p leaves a usable bank. Because evaluating equation (3.4) directly in the full feature dimension is expensive, the distance comparisons are performed on a low-dimensional random projection ψ that approximately preserves pairwise Euclidean distances (a Johnson-Lindenstrauss embedding [36]), exactly as in [28], while the selected vectors are stored at their original dimension. The result is a compact bank that fits in a few megabytes and enables sub-millisecond nearest-neighbour lookups, a prerequisite for edge deployment, while retaining near-uniform coverage of the normal-feature manifold.

Threshold calibration

Design space. Several threshold strategies are possible for a one-class detector: a fixed global value, a per-camera percentile, a ROC-optimal cut calibrated on labelled

anomalies, or a parametric rule derived from the distribution of normal scores. A fixed global threshold requires that the absolute scale of anomaly scores be consistent across cameras, which does not hold in practice - a plain opaque door scores structurally lower than a glass door with a visible outdoor background, so a threshold tuned on one would misfire on the other. A ROC-optimal threshold requires labelled anomalies, violating the one-class premise and risking overfitting to the appearance of the synthetic anomalies used for tuning. A per-camera percentile is more sensitive to outliers in the small calibration sample. The chosen strategy is a parametric rule that adapts to each scene using only normal data.

Parametric rule. For each of n_{cal} additional augmented variants of I_{ref} (the *calibration variants*, disjoint from the bank variants and generated with an independent random seed), the maximum patch anomaly score over the image is computed. These n_{cal} scores form the empirical distribution of the maximum anomaly score of a normal scene, with mean μ and standard deviation σ. The decision threshold is

$$\theta \;=\; \mu + k\,\sigma, \tag{3.5}$$

where k is a sensitivity hyperparameter. This is the rule of statistical process control (Shewhart charts): an event is flagged when it lies k standard deviations above the normal mean. Under a Gaussian assumption, $k = 3$ corresponds to a theoretical false-positive rate of about 0.13% and $k = 4$ to about 0.003%; the calibration scores are not exactly Gaussian (they are maxima of correlated patch scores), but k still provides a monotone, interpretable control over the false-positive/recall trade-off. It is the sole deployment-time hyperparameter and encodes the asymmetric cost of a missed obstruction relative to a false alarm: for safety-critical exits k is set low (2.5 to 3.0) to favour recall, and higher (4.0 to 5.0) where false alarms are operationally costly.

Coverage dependency. Because μ and σ are estimated on augmented variants, they reflect only the score variability those augmentations induce. If a real source of variation (cast shadows, light beams, outdoor background changes through glass) is absent from the augmentation scheme, σ is underestimated and real test scores under that variation exceed θ more often than expected. This dependency is the root cause of the shadow false-positive problem (Section 5.1) and motivates the shadow and light augmentation introduced below.

Test phase and anomaly map

At inference, the query image is processed by the same feature pipeline to produce its patch vectors. Each patch vector ϕ is compared to the memory bank through nearest-neighbour search; let m^* be its nearest bank neighbour, so that the raw patch distance is $s^*(\phi) = \|\phi - m^*\|_2$. Let $\mathcal{N}_b(m^*)$ be the set of b nearest neighbours

of m^* within the bank. Following equation 7 of [28], the patch score is re-weighted as

$$s(\phi) \;=\; w(\phi)\, s^*(\phi), \qquad w(\phi) \;=\; 1 - \frac{\exp\|\phi - m^*\|}{\sum_{m \in \mathcal{N}_b(m^*)} \exp\|\phi - m\|}. \tag{3.6}$$

Two points deserve care. First, the distances in the denominator are between the test patch ϕ and the members of $\mathcal{N}_b(m^*)$, *not* between m^* and its own neighbours - a detail required for fidelity to the original formulation. Second, because m^* is the nearest neighbour, every distance in the denominator is at least $\|\phi - m^*\|$, so the fraction is at most $1/b$ and the weight is bounded as $w(\phi) \in [\,1 - 1/b,\, 1)$. The re-weighting therefore never inflates a patch score above its raw distance s^*: it leaves $s(\phi)$ close to s^* when m^* is isolated (its neighbours are far, the denominator is large, $w \to 1$), and discounts it by up to a factor $1 - 1/b$ when m^* sits in a dense region of common normal patterns (its neighbours are close, $w \to 1 - 1/b$). In relative terms this promotes patches matched to rare normal prototypes - the configuration most likely to indicate a genuine anomaly.

The original paper assigns the *image*-level score to the single most anomalous patch, $\arg\max_\phi s^*(\phi)$, re-weighting only that one (equations 6-7 of [28]). The present work instead evaluates equation (3.6) for *every* patch, arranges the scores on the spatial grid to form an anomaly map, upsamples it bilinearly, and smooths it with a Gaussian filter; the scalar image score is then the maximum of this smoothed map, $s_{\max}$, and the binary decision is $s_{\max} > \theta$. This is a deliberate variant of the paper's single-patch score, adopted for two reasons: the full per-patch map is required by the spatial gating of Section 3.2.1, which operates on connected components of the map; and taking the maximum of the smoothed map - rather than of a single raw patch - is more robust to an isolated noisy patch, since smoothing broadens genuine anomaly blobs (helping small objects that span few patches) while attenuating one-pixel spikes. The two formulations coincide when no smoothing is applied and the maximiser of $w(\phi)\, s^*(\phi)$ is also the maximiser of s^*.

Shadow and light augmentation

The photometric-only memory contains no variants with cast shadows or localised light excess. This leaves a coverage gap: at test time, a shadow projected on the floor by a passer-by outside the field of view, or a light beam from a skylight or a glass door, produces patch features distant from all bank entries and can trip the threshold even though no obstruction is present. The two disturbances are symmetric - darkening versus brightening - and are addressed symmetrically by adding synthetic variants of each kind to the augmentation scheme. Three modalities are modelled for each (elliptical, directional, stripe), covering the physically plausible patterns in corridor and doorway scenes; all borders are blurred to avoid unrealistic high-frequency edges.

A disturbance is added to a build variant with probability p_{sh} (shadow) or p_{li} (light), and to a calibration variant with probability $p_{\text{sh}}^{\text{cal}}$ and $p_{\text{li}}^{\text{cal}}$ respectively. A

central design choice is to *decouple* the build and calibration probabilities, motivated by the different roles of the two phases:

- the **memory** is given a high disturbance probability ($p_{\text{sh}} = p_{\text{li}} = 0.4$), with the intent of covering disturbed patterns so that, at test time, shadow and light patches lie closer to bank entries;
- the **calibration** is given a low disturbance probability ($p_{\text{sh}}^{\text{cal}} = p_{\text{li}}^{\text{cal}} = 0.1$), to keep the score distribution used to estimate μ and σ stable.

The reason for the asymmetry is that raising the calibration probability to the build level would make σ high-variance - a few calibration variants with heavy disturbances would inflate σ, and the $k\sigma$ amplification would yield per-camera thresholds either too high (false negatives) or too low (false positives). The effectiveness of this design, and the relative contribution of the build and calibration phases, is assessed in Chapter 5. The two families are independent: a variant may receive a shadow, a light, both, or neither. The disturbances are deliberately kept out of the test images used to measure robustness, so that the reported false-positive rates reflect generalisation to disturbances never seen at build time.

Spatial gating via annotated polygons

Glass doors introduce a distinct failure mode: the transparent panel makes the external background visible, and any movement or lighting change beyond the door generates anomaly blobs in the glass region without any physical obstruction. The principled remedy operationalises the obstruction criterion defined for the test set (an obstruction reaches the door frame, Section 3.3.3) as a spatial constraint on the anomaly map.

Door-frame polygons (left jamb, right jamb, threshold, architrave) are annotated once on the reference image and rasterised into a binary *frame mask* F (Figure 3.2). The gating rule operates on the connected components of the thresholded anomaly map (Figure 3.3): the map is binarised at θ, its 8-connected components are labelled, and a component C is accepted only if

$$|C \cap F| \;\geq\; \tau, \tag{3.7}$$

where τ is the minimum overlap, in pixels, between a component and the frame mask. The final score is the maximum of the continuous anomaly map over the accepted components, or zero if none survives. A component confined to the glass panel has zero overlap with F and is rejected, whereas a floor-grounded obstruction produces a component reaching the threshold or jambs and is accepted. The same rule applies to opaque doors when the panel exhibits strong illumination variation: specular highlights stay on the panel, while grounded obstructions reach the frame.

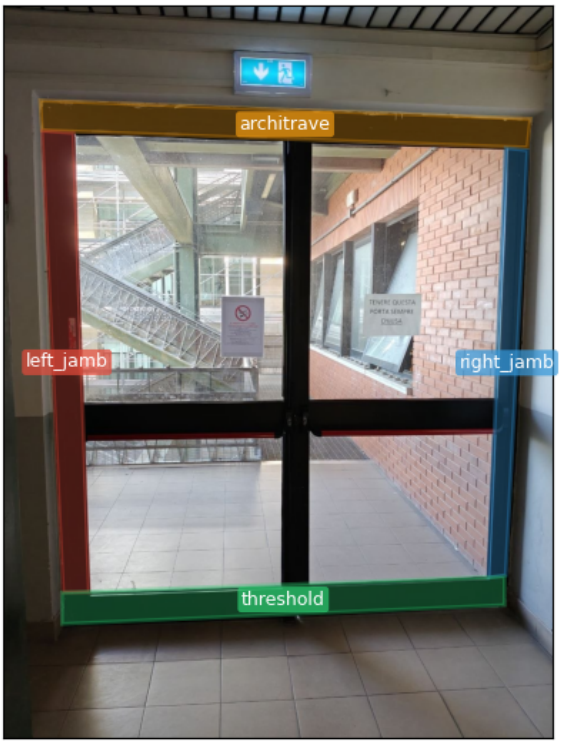


Figure 3.2: Door-frame polygons (architrave, left and right jambs, threshold) annotated once on a glass-door reference image and rasterised into the binary frame mask F used by the gating rule.

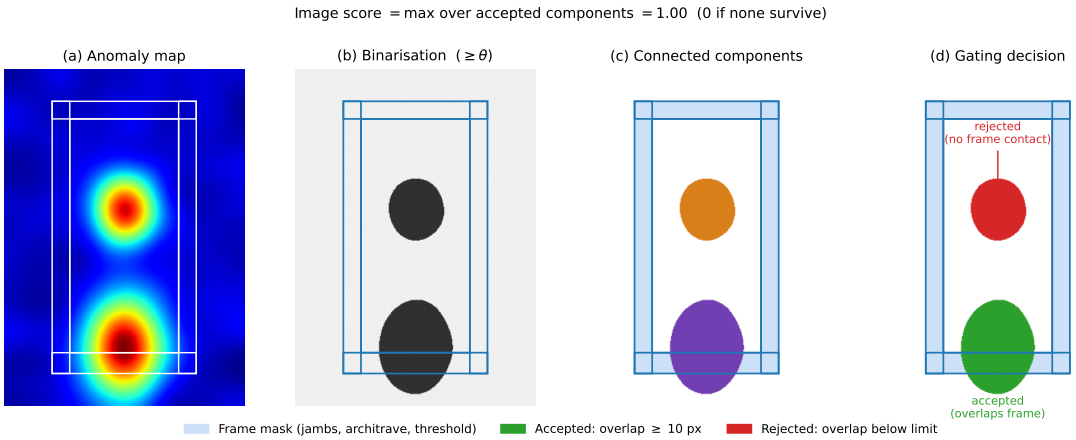


Figure 3.3: The connected-component gating rule, stage by stage. (a) The continuous anomaly map is (b) binarised at the per-camera threshold $\theta = \mu + k\sigma$; (c) the foreground is split into 8-connectivity connected components; (d) a component is a valid detection only if it overlaps the frame mask F (jambs, architrave, threshold) in at least τ pixels. The grounded obstruction reaches the threshold band and is accepted; a blob lying entirely on the door panel, which is excluded from F (e.g. the head of a thin-stemmed object), has no frame contact and is rejected. The image score is the maximum anomaly over the accepted components, or 0 if none survive. The anomaly map is schematic, but every downstream step runs the real algorithm.

Structural limitations. Gating assumes the frame geometry is stable and visible at test time. An *open door* violates this: the threshold and jambs change radically, producing an out-of-distribution component inside F (a structural false positive), while an obstacle in the open doorway no longer intersects the misplaced mask (a structural false negative). A thin-stemmed grounded object (a floor fan, a sign on a pole) has only a narrow contact with the frame that may fail the overlap criterion τ. These cases share one root: reachability of the frame mask by a genuine anomaly component depends on frame visibility, a property of the scene rather than of the model. They are declared as limitations of the method; the system is designed and validated for the nominal closed-door condition, dominant in continuous surveillance.

Calibration stabilisation: pooled sigma

The per-camera rule $\theta_i = \mu_i + k\,\sigma_i$ has a structural instability. The per-camera standard deviation σ_i is estimated from a small calibration sample and carries a relative error on the order of 18%; this error propagates one-to-one into the $k\sigma_i$ term, and across many cameras it produces a long-tailed distribution of thresholds - a few cameras receive thresholds high enough to miss all obstructions, others low enough to generate shadow false positives, even though the underlying scenes are not fundamentally different. The extreme σ_i values are stochastic calibration outliers, not scene outliers: the same camera re-calibrated with a different seed would receive a normal threshold.

The fix replaces the per-camera σ_i with a single population estimate while keeping μ_i local:

$$\theta_i \;=\; \mu_i + k\,\sigma_{\text{pop}}, \qquad \sigma_{\text{pop}} = \text{median}(\sigma_1, \ldots, \sigma_N), \tag{3.8}$$

where N is the number of cameras. The median is robust to the outlier σ_i that caused the tail. Keeping μ_i per-camera preserves scene-specific adaptation (the absolute score level of each scene), while pooling σ removes the noisy multiplicative term. Under the per-camera rule $\text{Var}(\theta_i) = \text{Var}(\mu_i) + k^2\,\text{Var}(\sigma_i) + 2k\,\text{Cov}(\mu_i, \sigma_i)$, whereas under the pooled rule it collapses to $\text{Var}(\theta_i) = \text{Var}(\mu_i)$, i.e. to the intrinsic scene variability alone.

Pooled sigma is, by construction, a shrinkage (regularisation) estimator: it trades a small bias - forcing each camera toward the population median - for a large variance reduction. In the data-starved regime of this thesis (a one-shot memory from a single image with sparse synthetic calibration) the per-camera σ_i is noise-dominated and the true cross-camera heterogeneity is near zero, so full pooling is the correct corner. The principled generalisation is *partial pooling*, $\sigma_i^* = (1 - w_i)\,\sigma_{\text{pop}} + w_i\,\sigma_i$, with the weight w_i rising as a camera accumulates real normal frames; full pooling ($w_i = 0$) is the cold-start corner and remains valuable for newly installed cameras.

3.2.2 Single-Class Obstruction Detection with YOLO (P1)

Motivation and design rationale

P1 approaches the same objective from the supervised side: instead of learning what the normal scene looks like and flagging deviations, it learns what an *obstacle* looks like and localises it directly. This inverts the trade-offs of P3. A detector is scene-agnostic - it requires no per-camera reference image, no calibration, and keeps working if the camera is moved - and it is intrinsically robust to the nuisance variations that challenge a reference-based method: a shadow, a light beam, or a background change behind a glass panel contains no object and therefore, in principle, triggers no detection. The price is the closed vocabulary: a detector can only find what it was trained to find, so an obstacle type absent from the training data is invisible to it. The two pipelines are thus natural complements, and their failure modes are expected to be disjoint - a hypothesis the shared real test set of Section 3.3.3 makes verifiable.

Two design decisions characterise P1. First, the task is cast as *single-class* detection: every obstacle category collapses onto one class, `obstruction`, because the operational question is whether the route is blocked, not by what. This concentrates the model capacity on the object-versus-background boundary rather than on inter-class discrimination, and it simplifies the decision rule at deployment. Second, before any training was undertaken, the *frozen-vocabulary hypothesis* was tested: whether a detector pre-trained on a generic corpus (MS-COCO [17]), used as-is with a suitable mapping of its classes onto the obstruction concept, could reach adequate accuracy at zero training cost. As Chapter 5 shows, this hypothesis fails - the obstacle categories of the evacuation domain are essentially absent from the COCO vocabulary - and its failure is what motivates the fine-tuning route developed here.

Key intuition

The core intuition mirrors, in supervised form, the transferability argument of P3: a detector pre-trained on a large natural-image corpus has already learned generic objectness - edges, textures, part-whole composition - and needs only to re-target its final layers to the specific appearance of evacuation-route obstacles. Fine-tuning from pre-trained weights therefore requires orders of magnitude fewer labelled images than training from scratch. The labelled data itself comes at near-zero manual cost: the copy-paste corpus (D1, Section 3.3) records the exact position of every pasted object, so bounding-box supervision is a by-product of the synthesis rather than a manual annotation effort. The open question - whether a detector trained on synthetic composites transfers to real scenes - is empirical, and Chapter 5 addresses it on two independent test sets.

One-stage detection: formulation

All the detectors considered in P1 belong to the one-stage family, which frames detection as dense regression over a feature grid rather than as a two-step propose-then-classify procedure [27]. The input image is processed by a convolutional *backbone* that produces feature maps at several scales; a *neck* fuses information across scales, so that both small and large objects are represented; and a *head* predicts, at every location x of each fused map, a classification score $p(x) \in [0, 1]$ for each class and a bounding box $b(x)$. Modern Ultralytics heads are *anchor-free*: instead of regressing offsets from a set of pre-defined anchor boxes, the head directly predicts the distances from the grid location to the four sides of the box, removing the anchor hyperparameters [15].

During training, each ground-truth box must be assigned to the grid locations responsible for predicting it. The assignment follows Task-Aligned Learning (TAL) [11]: for every candidate location the alignment metric

$$t \;=\; p^{\alpha} \cdot \mathrm{IoU}^{\beta} \tag{3.9}$$

combines the classification score p for the target class and the Intersection-over-Union IoU between the predicted and ground-truth boxes, with exponents α and β weighting the two terms. The top-scoring locations under t become positives for that object. Because t is high only when *both* the classification and the localisation are good, the assignment explicitly aligns the two sub-tasks - a location that classifies well but localises poorly is not reinforced. The loss combines binary cross-entropy for classification with an IoU-based loss for box regression; the generation-specific regression terms are described below.

At inference, a one-to-many head produces multiple overlapping candidate boxes per object, which are pruned by non-maximum suppression (NMS): boxes are ranked by confidence and any box whose IoU with an accepted box exceeds a threshold is discarded. NMS is a post-processing step external to the network; its removal is one of the axes along which the tested architectures differ.

YOLO11

YOLO11 [15] is the eleventh Ultralytics iteration of the YOLO lineage [26] and the reference CNN architecture of this study. Relative to its predecessor YOLOv8 it makes three structural changes: the backbone and neck adopt the `C3k2` block, a leaner formulation of the Cross-Stage-Partial bottleneck that splits one large convolution into two smaller ones; a `C2PSA` block - Cross-Stage-Partial with spatial attention - is inserted after the `SPPF` multi-scale pooling module, letting the network re-weight spatial positions before the neck; and the classification branch of the head uses depthwise-separable convolutions, cutting its cost. The nano variant used as the primary deployment candidate has 2.6 million parameters and 6.5 GFLOPs at 640×640 input. The same architecture scales to small (9.4M) and medium (20.1M)

variants by width and depth multipliers, a property exploited in Chapter 5 to isolate the effect of model capacity at fixed architecture.

YOLO26: end-to-end NMS-free detection

YOLO26 [35] is the most recent generation of the same family and differs from YOLO11 along one architectural axis and three training axes. Architecturally, it is *natively end-to-end*: a one-to-one detection head outputs at most one box per object directly, so NMS is no longer required - the same design goal pioneered by DETR [4], achieved here without a transformer. It also removes the Distribution Focal Loss (DFL) module from box regression; DFL discretises each box coordinate into a histogram over fixed bins, and its removal simplifies the head, removes the bin-range constraint on the regression, and eases export to edge runtimes. On the training side, YOLO26 introduces ProgLoss (a progressive re-weighting of the loss toward the end-to-end branch), Small-Target-Aware Label assignment (STAL, which preserves positive assignments for small objects), and the MuSGD optimiser, a hybrid of SGD and Muon-style updates. The nano variant has 2.4M parameters and 5.4 GFLOPs, with a claimed CPU-inference speed-up of up to 43% over YOLO11n [35] - a relevant property for the Raspberry Pi deployment target, which has no GPU.

RT-DETR: a transformer-based comparison point

To separate what is specific to the YOLO lineage from what any modern detector would achieve, the comparison includes RT-DETR [19], the first transformer-based detector designed for real-time use. Like DETR [4], it casts detection as *set prediction*: a fixed set of learned object queries attends to the image features in a transformer decoder, and training matches queries to ground-truth objects one-to-one by solving a bipartite (Hungarian) assignment on a combined classification-localisation cost. This makes the model NMS-free by construction. Its distinctive component is an *efficient hybrid encoder* that decouples the two expensive operations on multi-scale features: attention-based intra-scale interaction (AIFI) runs only on the deepest, smallest feature map, while cross-scale fusion (CCFM) is handled by lightweight convolutional blocks - the design that brings transformer detection within real-time budgets. Query initialisation is *IoU-aware*: the decoder starts from the encoder locations most likely to contain objects. The variant used here, RT-DETR-L, has 32.8M parameters and 108 GFLOPs - an order of magnitude above the nano CNNs - and serves as the CNN-versus-transformer axis of the comparison, not as a deployment candidate.

Training data and anti-leakage

The detector is trained on the synthetic corpus D1 exclusively; no real test frame contributes to training. Positives are the 990 copy-paste composites with their synthesis-derived boxes, all categories collapsed to the single `obstruction` class.

Negatives - images with no label file, which the Ultralytics loader treats as pure background - are the 110 reference backgrounds that the copy-paste generator deliberately left clean (Section 3.3). This choice is dictated by leakage: the backgrounds actually *used* by the composites cannot serve as negatives, because each of them also appears, with an object pasted on it, among the positives; a detector could then lower its loss by memorising the background identity instead of learning the object appearance. The held-out clean backgrounds share the visual distribution of the positives' scenes without sharing any specific image.

The same group-key discipline of Section 3.3.5 governs the cross-validation folds: splits are stratified by venue and label but *grouped* by the background identifier, so that no background - clean or composited - ever spans the training and validation side of a fold. A further exclusion, specific to the generative test set introduced in Section 3.3.4, removes from training every composite whose background also appears in that test set; its quantitative effect is measured in Chapter 5.

3.3 Dataset Description

The problem addressed in this thesis - deciding whether an evacuation route is obstructed - has no public benchmark. The anomaly-detection literature relies on industrial corpora such as MVTec-AD [1] or VisA, which contain neither safety doors nor real corridors, while the available object-detection datasets for the obstacles typical of a hospital setting - wheelchairs, carts, waste bins, or stretchers - target the recognition of the object *in itself*, not its presence as an obstacle in a specific scene. The data used here was therefore assembled from scratch, along two complementary regimes:

- a *synthetic construction corpus*, built by copy-paste composition, which provides a large and fully controlled body of normal and obstructed scenes for development and for the preliminary, controlled validation of the models;
- a *real on-site test set*, photographed in the field, which is the only genuine measure of operational performance and is shared, as a common benchmark, by all the pipelines compared in this thesis.

This section describes how the two corpora are constructed and from which sources; how each pipeline *consumes* the data (which split feeds the bank, the calibration, or the detector training) is detailed in the corresponding pipeline section.

3.3.1 Sources of raw material

Normal background scenes

The normal scenes - unobstructed safety doors and corridors - were collected by scraping Pinterest, one of the few sources offering a large variety of real indoor

scenes with heterogeneous lighting and viewpoints. The harvested images were filtered by hand to discard close-ups of handles or knobs, scenes already containing obstructions, photorealistic renders and obviously AI-generated pictures, and near-duplicates. The result is organised by *venue* (Table 3.1). Files are renamed to a canonical form (`porta_NNN`, `corridoio_NNN`); the file stem doubles as the *group key* later used by the database to tie a clean scene to all of its synthetic variants (Section 3.3.5). A small conversion utility normalises the encoding to JPEG, flattening any alpha channel onto a white background.

Venue	Images
Door	550
Corridor	550
Total	1100

Table 3.1: Normal background scenes scraped from Pinterest after manual filtering.

Obstacle objects

The obstacle categories most relevant to the evacuation scenario were obtained as five independent YOLO-format datasets from Roboflow: cart, wheelchair, box, stretcher, and waste bin. Each comes with images and bounding-box labels. The original per-dataset class identifiers are retained on disk, but at composition time all categories collapse onto a single class `obstacle`: for the task at hand the obstacle *type* is irrelevant, only its presence inside the area of interest matters.

3.3.2 Synthetic occlusion corpus (D1)

Object extraction

To paste an object onto an arbitrary background its original background must first be removed. Each labelled bounding box is cropped from its source image and passed to `rembg`, a matting tool based on the U^2-Net salient-object segmentation network [25], which returns a four-channel PNG with the subject isolated on a transparent canvas. The output is one transparent cut-out per instance. The step is imperfect on thin borders (cart wheels, stretcher straps), where a faint white halo can remain, and floor reflections are occasionally included in the crop; the extracted set was therefore filtered by hand to remove poorly segmented cut-outs.

Copy-paste composition

The obstructed images are produced by compositing, onto each background, one to three cut-outs sampled uniformly across the five categories. For every object a valid position and scale are searched (up to a fixed number of attempts), the object is alpha-blended onto the scene, and the resulting bounding box is recorded

in normalised YOLO coordinates. The placement constraints (Table 3.2) encode semantic assumptions about the problem: an object rests on the floor (its top edge cannot rise above mid-frame), occupies a plausible fraction of the frame height, and overlaps other objects only marginally, so that each instance remains distinguishable. A fixed share of backgrounds is deliberately left clean, to emit negative samples in the same output format.

A further subset of variants deliberately relaxes the placement constraints of Table 3.2 to introduce *partial occlusion*: objects are cropped by the frame edge or made to occlude one another substantially, reproducing the incomplete, fragmentary views common in a cluttered corridor. These variants target the detector-based pipeline (P1), which must localise obstacles from partial evidence; they leave P3 unaffected, as its reference-based scoring reacts to any departure from the normal scene irrespective of whether a single object is fully visible.

Parameter	Value	Rationale
Scale range	$[0.15, 0.50]$	Object height as a fraction of frame height
Min. top edge	0.50	Object grounded on the floor, no floating
Max. pairwise IoU	0.10	Each instance clearly separable
Placement attempts	30	Empirical cap before giving up
Clean-background ratio	0.10	Explicit negative samples

Table 3.2: Constraints of the copy-paste composition.

The output filename encodes the venue, the background index, and the variation (`porte_0042_porta_042_v01`); a regular expression in the loader recovers the source stem (`porta_042`) and uses it as the group key, so each synthetic image is provably linked to its originating background. The corpus comprises 990 composited images with their YOLO label files (495 per venue).

Limitations of the synthetic corpus

Copy-paste composition introduces, by construction, two simplifications that must be kept in mind when interpreting results. First, *illumination and colour are not coherent*: a cut-out carries the lighting of its source scene, which rarely matches the target background, so a method relying on edge micro-discontinuities may find the object easier than a real obstruction. Second, *cast shadows are absent*: the pasted object projects no shadow onto the new scene, removing a cue while a residual shadow from the source crop may sit in a semantically wrong place. Both effects are visible in the example of Figure 3.4. These limitations are precisely what motivates evaluating the pipelines on the real test set described in Section 3.3.3.

(a) Normal background (b) Copy-paste obstruction

Figure 3.4: A pair from the synthetic copy-paste corpus (D1): (a) a normal door background and (b) the same scene with two cut-outs - a waste bin and a wheelchair - alpha-blended in front of it. The pasted objects carry the lighting of their source images and cast no shadow on the floor, the two simplifications discussed in the text.

3.3.3 Real on-site test set (D2)

All the material above is either synthetic or web-sourced: it is suited to *building* and *validating* the models, but it does not represent a true operating condition. The final evaluation is therefore carried out on a second dataset, photographed directly in the field at the engineering campus of the University of Pisa. This set is the only real test of the thesis and is common to all pipelines: every method is compared on exactly the same real frames.

Acquisition and structure

Six distinct exits doors were photographed, all of venue type *door*, at a resolution of 960×1280. For each scene the acquisition protocol provides three kinds of frame: a single *reference* image - the expected normal scene, free of obstructions, which is the baseline consumed by the reference-based pipelines and the anchor for the door-frame annotation; several *unobstructed variants* of the same scene; and several *obstructed variants*, in which a real object is physically placed in front of the exit. The set is balanced - 30 normal against 30 obstructed frames - with the six references serving as baselines rather than test queries.

Scope and obstruction criterion

Two of the pipelines compared in this thesis are *door-centric* by construction: the door-baseline (P2) and PatchCore (P3) both reason about a stable, annotated door frame, and therefore apply in practice only to doors - a corridor offers no comparably stable geometric reference. The real test set is restricted to doors for this reason,

Scene	References	Unobstructed (hard neg.)	Obstructed (real)
door_1	1	3	3
door_2	1	3	3
door_3	1	9	9
door_4	1	9	9
door_5	1	3	3
door_6	1	3	3
Total	6	30	30

Table 3.3: Composition of the real D2 test set, per scene.

so that every pipeline is evaluated on the same frames and under the conditions for which the door-centric methods are defined.

This scope also fixes the operational definition of an obstruction used to label D2, a definition shared by all pipelines and grounded on a physical constraint: *any real obstacle that blocks the passage rests on the floor and therefore intersects the door frame*. A frame is consequently labelled *obstructed* when an object reaches the door opening - that is, when it overlaps at least one jamb or the threshold; an object that is present in the scene but does not reach the frame (a bag on a distant bench, a person walking past) does not obstruct *that* exit, and the frame stays *normal*. The pipelines differ only in how they *operationalise* this criterion - P3, for example, enforces it as a connected-component gating rule against the rasterised frame mask (Section 3.2.1) - but the underlying ground-truth notion of obstruction is common to all.

Hard negatives and real obstructions

The distinction between the two classes is designed to be realistic and non-trivial. The unobstructed frames are not mere re-shots of the reference: they are *hard negatives*, the same door captured under nuisance variations that change the scene *without* reaching the door opening, and therefore must not be flagged - an object visible behind the glass panel, a strong shadow cast on the leaf, a notice taped to the door, a person transiting. Their purpose is to measure the false-positive rate in the regime where the scene changes but the route stays clear. The obstructed frames, conversely, contain a real obstacle resting in front of the exit and touching the frame (a waste bin moved against the door, a chair, and so on), with lighting and cast shadows naturally coherent with the scene; they are far more representative of a real obstruction than the copy-paste composites. The set thus stresses both tails of the problem: the detection of genuine obstructions (recall) and the robustness to false alarms under legitimate scene variation (specificity). Figure 3.5 illustrates the three frame types on a single glass door, where the *same* waste bin appears first behind the glass - not reaching the opening, hence a hard negative - and then on the near side, touching the frame and blocking the passage.

(a) Reference (normal) (b) Hard negative (c) Real obstruction

Figure 3.5: The three frame types of the real test set (D2) on one glass door: (a) the clean reference; (b) a hard negative, where the waste bin is visible behind the glass on the walkway along with a poster on the right side; (c) a real obstruction, the same bin placed on the near side, touching the frame and blocking the passage. The contrast between (b) and (c) is exactly the obstruction criterion of Section 3.3.3.

3.3.4 Generative photorealistic test set (D3)

The copy-paste corpus D1 and the real set D2 leave a gap between them: D1 is large but visually naive (incoherent lighting, no cast shadows), while D2 is realistic but small (30 obstructed frames) and restricted to doors. A third corpus fills this gap for the detector pipeline. Normal background scenes are edited by a generative image model (`gemini-2.5-flash-image` [13]) instructed, with a fixed prompt and seed, to add one to three obstacles of the usual five categories *on the floor, blocking the passage*, as a minimal local edit: resolution, framing, camera angle, and every pixel outside the added objects must remain identical to the original photograph, while the objects themselves must be photorealistic, correctly scaled, and consistent with the scene's lighting and perspective - including cast shadows, the two cues D1 lacks by construction. A share of the prompts additionally requires one object to be partially cut off by the image border, mirroring the partial-visibility variants of D1.

The current release comprises 120 images (60 doors, 60 corridors), all positives, with 166 obstacle instances. Bounding boxes were annotated by hand, and each instance additionally carries a hand-assigned *category tag* (cart, wheelchair, stretcher, box, waste bin), which enables the per-category failure analysis of Chapter 5; an earlier release of 94 images with 187 instances was used for part of that analysis before the set was extended. Since the set contains no negative frames, it supports detection-level metrics (mAP, instance recall) rather than image-level classification metrics; the latter remain the province of D2.

One caveat is structural: the background scenes edited by the generative model are drawn from the same Pinterest pool that feeds the copy-paste corpus, so a model

trained on D1 has typically *seen* the test backgrounds - with a different pasted object, or clean. D3 therefore measures generalisation to new objects on familiar scenes, not to new scenes. Both mitigations are in place: the training set is purged of every composite sharing a background with D3 (Section 3.2.2), and the residual effect of this overlap is measured empirically in Chapter 5. D3 is a P1-specific evaluation asset and is kept outside the database index of the next subsection.

3.3.5 Database organisation and anti-leakage

The images on disk are organised by type and venue, but the pipelines need to query the data along other axes - by split, by obstacle category, by reference frame, by source group. These relations are kept in a single SQLite database that acts as the canonical index: every training and evaluation query goes through it, and the files on disk are referenced by relative path. Each physical image is one row in a `frames` table recording, among other fields, its venue, whether it is normal, the occlusion type (`none`, `synthetic_copypaste`, or `real`), its split, its provenance (`source`), the group key (`source_group`), and, for derived images, the reference frame they originate from. Bounding boxes (normalised YOLO coordinates, single `obstacle` class) live in a companion table and are populated only by the copy-paste composites; the real obstructions carry frame-level ground truth only.

The chief risk in a synthetic corpus is *leakage*: if the same background appears as a normal image in training and as an obstructed image in test, a reference-based pipeline would meet at test time information it already saw while building. The loader prevents this through the group key: every variant of a scene (its clean image and all its synthetic composites) inherits the same `source_group`, and the split is decided by hashing that key, so a whole group always lands in a single split regardless of load order or of how many variants are added later. In the current operating point the entire construction corpus (Pinterest backgrounds and copy-paste composites) is assigned to `train` and the real *poli ingegneria* set is the only `test` split; the *poli ingegneria* frames all share a single group, which keeps the real set permanently disjoint from the construction material. No fixed `val` split is materialised, since validation is performed by cross-validation over the training pool. Table 3.4 summarises the resulting inventory.

Source	Venue	Normal	Occlusion	Count
Pinterest (backgrounds)	door + corridor	yes	none	1100
Copy-paste composites (D1)	door + corridor	no	copy-paste	990
D2 (references)	door	yes	none	6
D2 (hard neg.)	door	yes	none	30
D2 (obstructed)	door	no	real	30
Total				2156

Table 3.4: Inventory of the dataset shared across the pipelines. The construction corpus (2090 frames) forms the `train` pool; the real *poli ingegneria* set (66 frames) is the held-out `test` split. The P3-specific shadow and light robustness variants are described in Chapter 4 and are not listed here.

Chapter 4

Experimental Setup

4.1 Pipeline Implementation

4.1.1 Unsupervised Anomaly Detection with PatchCore (P3)

This section reports the concrete technical settings of the method derived in Chapter 3. Each monitored exit is handled by an independent memory bank built from a single reference frame, reflecting the deployment reality of a fixed camera with a stable normal scene.

Backbone and feature extraction

The backbone is a WideResNet50-2 [37] pre-trained on ImageNet and used frozen (no fine-tuning). Every image is resized to 224×224 and normalised with ImageNet statistics. Features are read from blocks ℓ_2 and ℓ_3:

Block	Spatial resolution	Channels
ℓ_2	28×28	512
ℓ_3	14×14	1024

Table 4.1: Feature maps read from the frozen backbone at 224×224 input.

The ℓ_3 map is upsampled bilinearly to 28×28 and concatenated with ℓ_2 along the channel axis, giving a tensor of shape $[1536, 28, 28]$. The 3×3 neighbourhood average pooling of equation (3.1) is applied with unit stride and same-padding, and the result is flattened to $N_p = 784$ patch vectors of dimension 1536 per image. The shallowest block (56×56, low-level texture) and the deepest block (7×7, class-biased abstraction) are not used.

Build and calibration settings

The bank is built from $n_{\text{aug}} = 15$ photometric variants of I_{ref} (random seed 42); the original reference is excluded. The photometric augmentation ranges are listed in

Table 4.2.

Transform	Range	Rationale
Brightness	$\times[0.85, 1.15]$	Morning / evening / artificial light
Contrast	$\times[0.90, 1.10]$	Direct vs. diffuse illumination
Saturation	$\times[0.90, 1.10]$	LED / fluorescent / incandescent
Red-channel shift	$[-8, +8]$	Colour temperature
Blue-channel shift	$[-20, +20]$	Colour temperature
Gaussian noise	$\sigma \in [1, 4]$	Sensor noise
Gaussian blur	$r \in [0.5, 1.5]$, $p = 0.3$	Slight lens defocus

Table 4.2: Photometric augmentation applied at build time. No large geometric transform is used, as the camera is fixed.

In addition to the photometric transforms, an optional small geometric perturbation is available for cameras subject to micro-vibration, disabled by default ($\delta_{\max} = 0$, $\rho_{\max} = 0$). When enabled, every variant receives a per-variant random translation drawn uniformly in $[-\delta_{\max}, +\delta_{\max}]$ pixels on each axis and a random rotation drawn uniformly in $[-\rho_{\max}, +\rho_{\max}]$ degrees, applied by bilinear affine warping at the original resolution; typical settings are $\delta_{\max} = 8$ px and $\rho_{\max} = 2°$. Unlike the forbidden flips and wide rotations, this jitter perturbs the whole scene rigidly and teaches the memory to tolerate a small global displacement at no inference-time cost. It is left off for all experiments reported in Chapter 5, where the data-collection protocol (fixed foot position, braced camera) keeps hand-held variation within the native tolerance of the neighbourhood pooling and the anomaly-map smoothing.

The raw memory of $n_{\text{aug}} \cdot N_p = 15 \times 784 = 11760$ vectors is reduced by greedy coreset selection to a fraction $p = 0.01$ (retaining at least 50 points), leaving about 117 vectors. For efficiency the greedy selection of equation (3.4) runs on a sparse random projection to 128 dimensions (a Johnson-Lindenstrauss embedding), while the final coreset is taken from the original 1536-dimensional features.

The threshold is calibrated on $n_{\text{cal}} = 10$ further variants generated with seed 1000, disjoint from the bank seed to guarantee statistical independence; later experiments raise this to $n_{\text{cal}} = 30$ to reduce the sampling error on σ. The re-weighting of equation (3.6) uses a neighbourhood of size $b = 9$, and the anomaly map is smoothed with a Gaussian filter of $\sigma = 4$. The memory bank is serialised to a single file holding the coreset features together with the calibration parameters μ and σ.

Anti-leakage separation

The bank, calibration, and test sets are strictly disjoint (Table 4.3): no data used to build the memory or calibrate the threshold enters the test phase, and I_{ref} is reserved as the only genuinely unseen normal sample.

Set	Seed	Contains I_{ref}?	Role
Bank variants ($n_{\text{aug}} = 15$)	42	No	Builds the feature memory
Calibration variants ($n_{\text{cal}} = 10$)	1000	No	Estimates μ and σ
Normal test	–	Yes	Independent normal sample
Obstructed test	–	–	Anomalous query

Table 4.3: Disjoint partition of data across build, calibration, and test.

Shadow and light augmentation

Both disturbance families implement three modalities, summarised in Table 4.4. They are applied after the photometric transforms, stacking $k_s \in [1, 3]$ instances per affected variant. The light family adds two axes absent from shadows: a diagonal (solar) stripe parameterised by a continuous incidence angle, and a warm colour shift (red up, blue down) on the elliptical and diagonal-stripe lights, modelling the chromaticity of direct sunlight. As a consequence the light subspace is intrinsically richer, so the raw values of light and shadow false-positive rates are not directly comparable; only within-category comparisons (across configurations) are legitimate.

Modality	Shadow	Light
Elliptical	dark ellipse, blurred border	bright ellipse (beam, specular)
Directional	dark edge gradient	bright edge gradient (lateral window)
Stripe	dark band (2 orientations)	bright band (3 orient., solar diagonal)
Colour shift	none	warm tint on elliptical / diagonal

Table 4.4: Synthetic disturbance modalities. Build probabilities $p_{\text{sh}}, p_{\text{li}} = 0.4$; calibration probabilities $p_{\text{sh}}^{\text{cal}}, p_{\text{li}}^{\text{cal}} = 0.1$.

To formally separate the test-time disturbances from the build-time ones, the build augmentation draws its shadows and lights from a dedicated random generator seeded disjointly from the generator that produces the test disturbances, so the two sets never coincide.

ROI annotation and connected-component gating

The frame mask of equation (3.7) is produced by a three-stage tool chain. A lightweight interactive annotator draws one polygon per frame element (left jamb, right jamb, central mullion, threshold, architrave, or a catch-all label) and writes one JSON file per image, storing polygons in original-image coordinates to avoid precision loss after resizing. An importer loads these polygons into the database, matching by relative file path. At evaluation time the polygon set of the reference frame is rasterised to the original resolution, the smoothed anomaly map is binarised at θ_i, its 8-connected components are extracted, and a component is kept only if

its overlap with the frame mask reaches the minimum $\tau = 10$ pixels. The score is the maximum of the continuous map over the surviving components, or zero if none survive.

The binarisation threshold used to form the components is the same per-camera θ_i used for the decision, so the sensitivity k acts at two stages - which pixels survive binarisation, and which final scores clear the decision. A legacy rectangular ROI can be composed in front of the polygon gating: the rectangle zeroes the map outside the door bounding box first (suppressing floor reflections and solar shadows), and the polygon rule then filters the remaining glass-panel variation. The full image is always included in the memory bank: excluding the glass region would leave its patches without any neighbour at test time, inflating their distance by mere absence of reference and making the panel anomalous by construction.

Pooled-sigma evaluation

The pooled threshold of equation (3.8) is computed in two passes over the cameras. The first pass builds each bank, calibrates μ_i and σ_i, and scores all test samples without yet deciding their label. The second pass collects the σ_i, sets σ_{pop} to their median, and recomputes for every sample the threshold $\theta_i = \mu_i + k\,\sigma_{\text{pop}}$, the normalised score $(s - \mu_i)/\sigma_{\text{pop}}$, and the binary decision. The normalisation makes scores comparable across cameras of different absolute scale, so that the aggregated AUROC measures intrinsic separability rather than scale differences. The mode (per-camera or pooled) and the value of σ_{pop} are recorded with the experiment for reproducibility. Pooling affects neither inference time nor memory size, since calibration variants never enter the coreset; its only cost is a one-off computation at build time.

4.1.2 Single-Class Obstruction Detection with YOLO (P1)

This section reports the concrete settings of the detector pipeline derived in Section 3.2.2. All experiments use the Ultralytics framework (version 8.4.89) on PyTorch 2.5.1 with CUDA 12.4; training runs on a remote NVIDIA A100-SXM4-80GB GPU, while all evaluations run locally on CPU, a deliberate check that inference does not require server-class hardware.

Models

Seven detectors are fine-tuned, spanning two axes: *capacity* within a fixed architecture (nano, small, medium) and *architecture* at comparable scale (CNN versus transformer). All start from COCO-pretrained weights (Table 4.5).

A note on the memory budget is due, since the deployment hardware (Raspberry Pi or Jetson Nano) hosts the detector alongside the P3 memory-bank pipeline and a pose-estimation model. Measured at rest, the truncated P3 backbone (stem and blocks ℓ_1–ℓ_3 of WideResNet50-2) occupies about 100 MB, against 5 MB for

Model	Type	Parameters	GFLOPs	Weight file
YOLO11n	CNN, one-to-many + NMS	2.6M	6.5	5.3 MB
YOLO11s	CNN, one-to-many + NMS	9.4M	21.5	18 MB
YOLO11m	CNN, one-to-many + NMS	20.1M	68.0	39 MB
YOLO26n	CNN, end-to-end	2.4M	5.4	5.2 MB
YOLO26s	CNN, end-to-end	9.9M	–	19 MB
YOLO26m	CNN, end-to-end	21.8M	–	42 MB
RT-DETR-L	Transformer, end-to-end	32.8M	108.0	64 MB

Table 4.5: Detectors fine-tuned in P1. The nano CNNs are the deployment candidates; the larger variants isolate the capacity axis, and RT-DETR-L the architecture axis.

YOLO11n: the detector is not the memory bottleneck of the stack, so the choice among the nano, small, and medium variants is driven by accuracy results (Chapter 5), not by RAM.

Training configuration

All models share one configuration; nothing is tuned per-model, so that any performance difference is attributable to the architecture alone. Training runs for up to 150 epochs with early stopping when the validation fitness does not improve for 20 consecutive epochs, at input resolution 640×640, batch size 32 (fixed, rather than memory-adaptive, for reproducibility on a shared GPU), and seed 42. The single-class mode of the framework collapses all box labels onto one class at load time. The optimiser is the framework's automatic choice, and the learning-rate schedule is the default linear decay - one consequence, verified empirically, is that the schedule is coupled to the epoch budget, so intermediate checkpoints of a longer run are not equivalent to a shorter run.

Data augmentation keeps the Ultralytics defaults (Table 4.6). Two defaults deserve comment: vertical flip is disabled, which is semantically correct for upright indoor scenes; and the framework's own copy-paste augmentation stays off, as it requires segmentation masks that the box-only labels do not provide.

Numerical stability of RT-DETR. The only per-model deviation concerns RT-DETR, whose training with the default automatic mixed precision (AMP) is numerically unstable: the official trainer documentation warns that "AMP training can lead to NaN outputs and may produce errors during bipartite graph matching" [34]. RT-DETR is therefore trained with AMP disabled (full fp32), all other settings unchanged; the collapse observed with AMP on, and its resolution by this single-variable change, are documented in Chapter 5. AMP stays on for the YOLO models, which train without anomalies.

Transform	Value	Note
Mosaic	1.0	Disabled for the last 10 epochs
HSV jitter (h/s/v)	0.015 / 0.7 / 0.4	Photometric variation
Translation	$\pm 10\%$	Random affine
Scale	$\pm 50\%$	Random affine
Horizontal flip	$p = 0.5$	Valid for indoor scenes
Vertical flip	$p = 0$	Kept off (upright scenes)
Random erasing	0.4	Occlusion robustness
Mixup / copy-paste	0	Off (default; no masks available)

Table 4.6: Augmentation at training time (Ultralytics defaults).

Anti-leakage: exclusion of shared backgrounds

Following Section 3.2.2, the training list excludes every image whose background stem appears in the generative test set D3: 107 positives and 13 clean negatives are dropped, leaving 883 positives and 97 negatives. The exclusion list is materialised as a versioned text file of background stems and applied at fold-construction time, with a hard failure if the list matches zero images - a guard against silently ineffective exclusion when the script runs on a machine where stems differ.

Evaluation protocol

P1 is evaluated at two levels, on two disjoint test beds.

Detection level (D3). Localisation quality is measured by mean Average Precision on the generative test set: mAP@50, the area under the precision-recall curve with a match declared at IoU ≥ 0.5 between predicted and ground-truth box, and mAP@50–95, its average over match thresholds from 0.5 to 0.95 - the stricter the threshold, the more the metric rewards precise localisation. One methodological detail matters: the confidence threshold must *not* be raised during mAP computation (the framework default retains virtually all candidates at $\tau_{\text{conf}} = 0.001$), because the precision-recall curve is traced by sweeping confidence, and discarding low-confidence detections beforehand truncates the curve and underestimates the metric. The reported precision and recall are taken at the confidence that maximises F1 along that curve.

Image level (D2). Operational accuracy is measured on the real test set with the same binary protocol as P3, making the two pipelines directly comparable: a frame is predicted *obstructed* when at least one detection with confidence $\geq \tau_{\text{conf}} = 0.25$ and box area of at least 1% of the image survives; accuracy, precision, recall, F1, and false-positive rate follow from the 2×2 confusion matrix over the 30 obstructed and 36 unobstructed frames. Per-image predictions are recorded to support the error analysis of Chapter 5.

For the frozen-vocabulary baselines of Chapter 5 (COCO models used without fine-tuning), a detection counts toward the obstructed decision only if its class belongs to a curated mapping of COCO classes onto the obstruction concept; the mapping and its refinement are described together with the results.

4.2 Application

[General description of the application, its modules, and overall architecture - to be completed.]

Chapter 5

Experimental Analysis

5.1 Pipeline Results

5.1.1 Unsupervised Anomaly Detection with PatchCore (P3)

Evaluation protocol

All results are obtained through a database-driven evaluation that, for each normal reference frame, (i) builds the memory bank from augmented variants of the reference (excluding the original), (ii) calibrates the threshold on independent variants, (iii) scores the original reference as the normal test sample, and (iv) scores the obstructed frames linked to that reference. Scores are normalised per camera as $(s - \mu_i)/\sigma_i$ before aggregating the AUROC, while threshold-based metrics (false-positive rate FPR, true-positive rate TPR, and F_1) use the per-camera threshold θ_i so that each binary decision reflects its own calibration.

The evaluation is organised in two macro-phases (Table 5.1), reflecting the two roles of the datasets introduced in Chapter 3. **Phase 1** runs on the synthetic copy-paste corpus (D1, split *test*, doors and corridors together; about 990 reference frames, each with one or more obstructed variants). Because a copy-paste obstruction leaves the background unchanged, only the pasted object is anomalous and the discrimination problem is near-saturated (AUROC ≈ 1); Phase 1 is therefore not a measure of headline accuracy but a *calibration study* - how to build the memory bank, place the decision threshold, and add shadow and light augmentation so that a single operating point stays robust to non-obstacle disturbances. Its internal structure is a progressive ablation, each component introduced in response to a failure mode identified empirically. **Phase 2** runs on the real on-site test set (D2): it measures operational accuracy on real doors with manually annotated gating, and here the reported metrics - recall, precision, F_1, accuracy - are the ones that matter.

Phase 1: calibration on the synthetic corpus (D1)

On D1 the discriminative problem is essentially solved before any tuning, so the goal of this phase is to convert the latent separability into a single, robust operating

Phase	Dataset	Focus	Key metrics
1	D1 (synthetic copy-paste)	Calibration: memory bank, decision threshold, shadow/light augmentation	FPR, threshold stability
2	D2 (real, on-site)	Real-world accuracy with manual polygon gating	Recall, precision, F_1, accuracy

Table 5.1: The two macro-phases of the P3 evaluation.

point. The ablation introduces one component at a time, each in response to a failure mode observed in the previous step.

Baseline. The baseline ($n_{\text{aug}} = 15$, $n_{\text{cal}} = 10$, no disturbance augmentation, $k = 3$) cleanly separates normal from obstructed scenes (Table 5.2). The near-perfect AUROC and the gap of several standard deviations between the class means confirm that obstacle patches fall well outside the distribution of normal door and corridor patches. The residual 0.5% FPR corresponds to about five false alarms over 990 frames. Discrimination is thus not the bottleneck; what remains is to keep this operating point stable once realistic disturbances enter the scene.

Category	Mean score	Std	Rate
Normal	2.309	0.156	FPR 0.5%
Obstructed	4.648	0.348	TPR 100%
AUROC		≈ 1.000	

Table 5.2: Phase 1 baseline on the synthetic test set.

Shadow vulnerability. A test set of shadow-augmented normal scenes (one synthetic variant per reference, $k_s \in [1, 3]$ stacked shadows) is scored against the baseline model. These are normal scenes with no obstruction, yet 90.1% are flagged as anomalous (Table 5.3). Shadow patches - dark, smoothly-bordered regions - are out of distribution for a bank built from shadow-free photometric variants, so they receive high anomaly scores, high enough to exceed a threshold calibrated on clean scenes.

The shadow scores (mean 3.19) land above the clean normals (mean 2.31) but still below the obstructions (mean 4.65): a shadow is more anomalous than a clean scene, yet clearly less than a real obstruction. The 90% FPR therefore comes from the decision threshold - calibrated on clean variants alone, it sits at ≈ 2.47, below the entire shadow band - and not from any inability to tell shadows from obstructions, a distinction made precise later (Section “Discrimination versus calibration”).

Category	Mean	Std	Range	Rate
Normal	2.309	0.156	$[1.87, 3.13]$	FPR 0.5%
Shadow normal	3.189	0.495	$[2.10, 4.51]$	FPR 90.1%
Obstructed	4.648	0.348	$[3.54, 5.87]$	TPR 100%

Table 5.3: Phase 1, shadow vulnerability: shadow-normal scores overlap the obstructed range $[3.54, 4.51]$.

Shadow augmentation and calibration stabilisation. Introducing shadow augmentation in the build phase, then decoupling and pooling the calibration, traces a clear path (Table 5.4). Coupling the build and calibration probabilities ($p_{\text{sh}} = p_{\text{sh}}^{\text{cal}} = 0.4$) reduces shadow FPR by two orders of magnitude but over-inflates the threshold, dropping TPR to 70.6%; the cause is- [ ] Aggiungere al dataset copy-paste varianti con oggetti tagliati a metà dalla scena. a long tail in the per-camera threshold distribution, where 12% of cameras generate 86% of all false negatives. Decoupling ($p_{\text{sh}}^{\text{cal}} = 0.1$, $n_{\text{cal}} = 30$) recovers TPR but swings to an over-permissive operating point. Replacing the per-camera σ_i with the pooled median $\sigma_{\text{pop}} = 0.300$ at $k = 4$ eliminates the residual threshold variance and reaches the best operating point.

Configuration	Clean FPR	Shadow FPR	TPR	F_1	σ_θ
Baseline (no aug)	0.5%	90.1%	100%	0.688	0.158
Coupled ($k = 3$)	0.0%	2.1%	70.6%	0.818	0.539
Decoupled ($k = 3$)	0.0%	28.4%	99.4%	0.873	0.388
Decoupled ($k = 4$)	0.0%	13.5%	93.4%	0.903	0.497
Pooled σ ($k = 4$)	**0.0%**	**12.2%**	**99.8%**	**0.941**	**0.137**

Table 5.4: Phase 1, shadow augmentation and calibration stabilisation. σ_θ is the standard deviation of the per-camera thresholds.

Pooling collapses the threshold range from $[2.28, 6.09]$ to $[3.24, 4.16]$ (a 73% reduction in σ_θ) and removes the pathological tail entirely: no camera exceeds $\theta = 4.16$ and TPR stays at 99.3% or higher in every threshold bucket. The 3.8 percentage-point F_1 gain is due entirely to variance reduction, not to any change in the backbone, whose AUROC stays at 0.998. The choice $k = 4$ follows a conservative four-sigma rule and was not tuned on the test set: the F_1-optimal value is about 3.9, but reporting it would constitute test-set tuning.

Light augmentation ablation. The symmetric counterpart of the shadow failure mode - localised brightness excess - is evaluated on a light-augmented test set ($\approx$ 1100 normal scenes) at $k = 4$, pooled σ, $n_{\text{cal}} = 30$. Five configurations are compared (Table 5.5), where the *budget* $p_{\text{sh}} + p_{\text{li}}$ is the expected disturbance probability per variant, a proxy for total augmentation intensity. A fifth fair-budget control (L4)

disentangles two effects that L2/L3 confound: increased coverage (adding the light type) and increased total intensity.

Config	p_{sh}	p_{li}	Budget	Shadow FPR	Light FPR	F_1
L0 shadow-only	0.4	0.0	0.40	12.2%	5.8%	0.917
L1 light-only	0.0	0.4	0.40	37.7%	23.2%	0.767
L2 SL-15	0.4	0.4	0.80	**2.2%**	**0.30%**	**0.982**
L3 SL-25	0.4	0.4	0.80	1.1%	0.10%	0.982
L4 fair-budget	0.23	0.23	0.46	11.2%	4.6%	0.929

Table 5.5: Phase 1 light-augmentation ablation. TPR is $\geq$ 97.7% throughout. L0/L2 use $n_{\text{aug}} = 15$, L3 uses $n_{\text{aug}} = 25$.

Four findings emerge. First, *light is a milder disturbance than shadow*: even with no light coverage (L0) the light FPR is 5.8%, below the 12.2% shadow FPR, because a light beam pushes features less out of distribution than a cast shadow. Second, *adding light augmentation works and helps shadows too*: from L0 to L2 the light FPR falls about nineteen-fold while the shadow FPR also improves rather than degrading. Third, L1 confirms that shadow augmentation is *structural, not incidental*: removing it collapses the per-camera threshold and inflates all FPRs, light included - light augmentation is a complement to shadow augmentation, not a substitute. Fourth, the fair-budget control shows that *the headline gain is mostly a budget effect*: L2/L3 apply twice the augmentation of L0, and the mean threshold grows monotonically with budget, tracking the FP reduction; at equal budget (L4 vs. L0) diversification alone yields only a small, cost-free improvement. The recommended configuration is SL-15 ($F_1 = 0.982$, light FPR 0.30%, shadow FPR 2.2%, recall 99.0%); SL-25 halves the FPRs again but trades recall for identical F_1 at a markedly longer build time, justified only when minimal FP is required. As established in Chapter 4, light and shadow FPR are not directly comparable; only within-category readings are valid.

Discrimination versus calibration. A cross-cutting question determines what the pipeline can legitimately claim: do the interventions improve the model's *discriminative capacity* or only the *placement of the threshold*? The clean metric is the per-camera AUROC - the probability that an obstruction scores higher than a normal variant of the *same* camera, on raw scores, averaged over cameras. Within a camera μ_i and σ_i are constant, so raw and normalised scores differ by an affine transform that preserves ranking; the per-camera AUROC is therefore invariant to both threshold and normalisation and isolates intrinsic discrimination.

The per-camera AUROC is saturated and nearly invariant to augmentation (Table 5.6): clean separation is perfect everywhere, the shadow value stays at 0.996 across the whole path - even with no shadow augmentation - and the light value is likewise saturated (0.997 to 1.000). The shadow FPR meanwhile swings from 2.1% to 90% to 12% while the raw AUROC is fixed at 0.9981. The mechanism is a *threshold shift, not a re-ordering*: the score distributions themselves barely move

Config	raw AUROC	per-cam AUROC (shadow)	shadow FPR	TPR	F_1
no-aug	0.9969	0.996	90.1%	100%	0.688
shadow04	0.9981	0.996	2.1%	70.6%	0.818
decoupled	0.9981	0.996	28.4%	99.4%	0.873
decoupled $k = 4$	0.9981	0.996	13.5%	93.4%	0.903
pooled	0.9981	0.996	12.2%	99.8%	0.941

Table 5.6: Controlled comparison: shadow FPR swings 40× while raw and per-camera AUROC stay fixed.

(shadowed-normal mean 3.19 → 3.14, obstructed 4.65 → 4.66), so within each scene an obstruction still outscores a shadow exactly as before and the per-camera AUROC is unchanged. What moves is the decision threshold $\theta_i = \mu_i + k\sigma$: calibrated on clean variants alone it sits low (mean ≈ 2.47), *below* the shadow band, so 90% of shadows exceed it; once disturbance variants enter the calibration set (raising μ and σ) and σ is pooled, θ rises (mean ≈ 3.74) *above* the shadow band but still below the obstructions, cutting the shadow FPR to ≈ 12% (Figure 5.1). None of this adds discriminative power - it places a single threshold so that the already-excellent latent discrimination becomes usable.

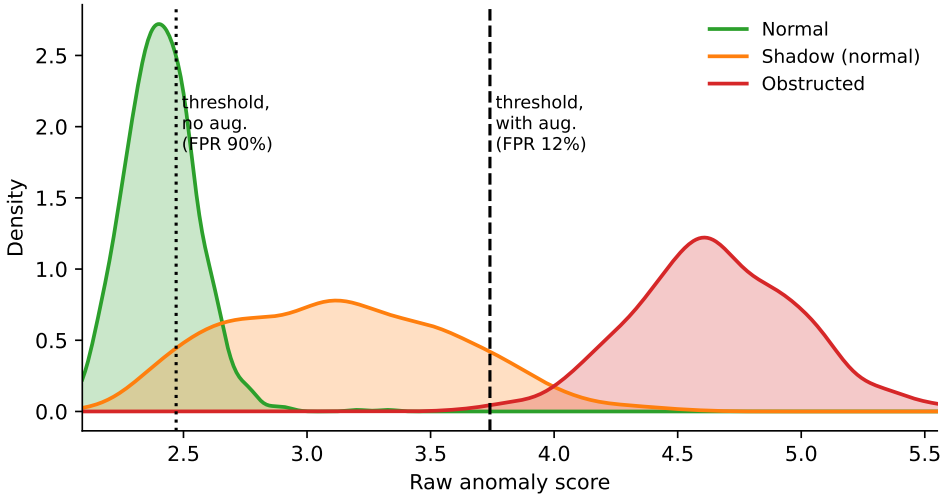


Figure 5.1: Calibration, not discrimination. The raw score distributions of normal, shadowed-normal, and obstructed scenes are essentially unchanged by augmentation, so their overlap - and the per-camera ranking (AUROC 0.996) - is fixed. Only the decision threshold moves: calibrated on clean variants it sits at ≈ 2.47 (dotted), inside the shadow band, flagging 90% of shadows; with disturbance-augmented, pooled calibration it rises to ≈ 3.74 (dashed), above the shadow band but still below the obstructions, cutting the shadow FPR to ≈ 12%. The thresholds shown are means of the per-camera values.

This conclusion holds on a test set where discrimination is near-saturated by

design (copy-paste obstructions on the same background: only the object is anomalous, a strong signal, AUROC ≈ 1). On harder data - real obstructions, variable backgrounds - the absolute AUROC values could differ; the per-camera AUROC argument is a claim about this dataset's regime and is stated as such.

Phase 2: real-world accuracy with manual gating (D2)

Phase 2 is the first evaluation on real photographs rather than copy-paste composites, and the first to report operational accuracy. The test set is D2: six real doors photographed on site (a mix of opaque and glass doors), each with one manually annotated reference frame, comprising 30 real obstructed images, 30 unobstructed images, and the six references. Consistent with the fixed-camera assumption, all shots of a door share the same framing, so the negatives measure the FPR under normal content and illumination variability at fixed geometry. The full pipeline is active: pooled σ plus connected-component gating ($\tau = 10$) against the hand-annotated door-frame polygons (the gating rule is illustrated stage by stage in Figure 3.3).

Validated on the synthetic suites, $k = 4$ proved too conservative on real scenes; lowering it to $k = 3$ raises recall at no FPR cost (Table 5.7; Figure 5.2). Because the pooled median(σ) = 0.213 is a shared constant, lowering k shifts every threshold down by the same amount without widening the distribution - the per-camera variance amplification of the naive rule does not reappear.

Metric	$k = 4$	$k = 3$	Δ
Recall (TPR)	83.3%	**90.0%**	+6.7 pp
FPR	3.3%	3.3%	invariant
Precision	96.2%	96.4%	+0.2 pp
F_1	0.893	**0.931**	+0.038
Accuracy	90.0%	93.3%	+3.3 pp

Table 5.7: Phase 2: recalibration of k on the real test set.

Comparing the two runs separates the false negatives by cause (Figure 5.3). Two are *calibration-limited* and are recovered at $k = 3$ through the two distinct stages where k acts: one is a decision-stage miss (a valid gated component scored just below the decision threshold at $k = 4$), the other a binarisation-stage miss (the obstruction's anomaly peak fell just below the local binarisation threshold at $k = 4$, leaving an empty binary map). The latter shows that θ controls not only the decision but which pixels exist for gating to evaluate, so an overly high k can erase a low-contrast obstruction before gating sees it. Three remain zeroed even at $k = 3$ and are *gating-limited*: the prototypical case is a free-standing floor fan whose wide head sits on the door panel (excluded from the frame mask by design) while its thin pole is the only part touching the frame and threshold (Figure 5.4). The pole is too slender to register as anomalous: only the head produces a strong response, so the fan does not form a single connected anomaly component that bridges the panel to

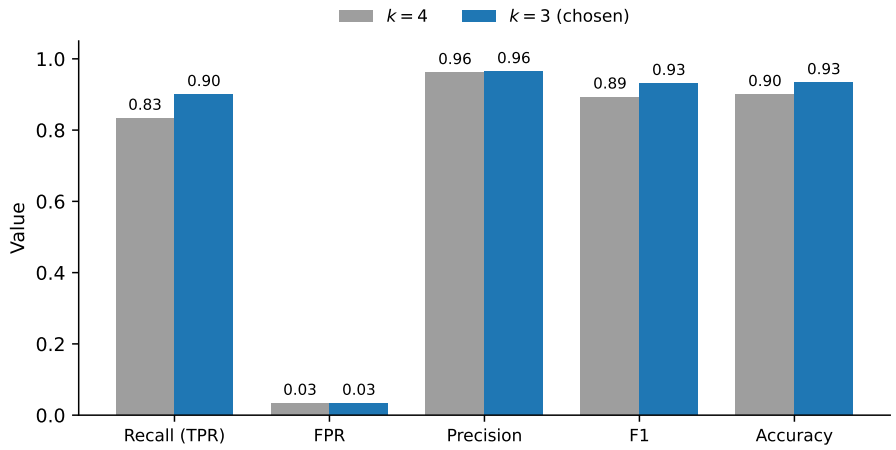


Figure 5.2: Operating-point metrics on the real test set (D2), at the synthetic-validated $k = 4$ and the recalibrated $k = 3$. Lowering k raises recall (83.3% $\rightarrow$ 90.0%), F_1 (0.893 $\rightarrow$ 0.931) and accuracy (90.0% $\rightarrow$ 93.3%) at unchanged FPR (3.3%) and precision.

a frame region. The head's component lies entirely in the excluded panel area and has no anomalous link reaching the frame, so gating zeros it. PatchCore detects the obstruction correctly; the failure is purely geometric, confirming the structural limitation anticipated in Chapter 3: the grounded-obstacle assumption holds for wide-profile objects (carts, wheelchairs, bins) but not for thin-stemmed ones (fans, coat racks, sign posts). A single FP is a glass door whose background changed behind the panel (Figure 5.5): with the external rolling shutter lowered, the view through the glass differs entirely from the reference frame, so the whole panel area lights up in the anomaly map and the pooled threshold is exceeded - a calibration limit of the fixed-background assumption rather than a gating failure. With order-of-tens samples, these numbers are a qualitative validation of the complete pipeline, not a production benchmark.

Discussion

Across the two phases, the failure modes of the method are diagnosable and fixable within the PatchCore framework without architectural changes. Cast shadows are addressed by disturbance-augmented, decoupled calibration that lifts the per-camera threshold above the shadow band (shadow FPR 90.1% $\rightarrow$ 2.2% at SL-15); localised brightness excess is addressed symmetrically with light variants; glass-door backgrounds are addressed by connected-component gating exploiting the floor-grounding constraint; and threshold instability across cameras is addressed by pooling the σ estimate. The unifying finding is that none of these fixes change the backbone's discriminative capacity, which is saturated; the bottleneck is placing a single threshold robust to disturbances.

The conclusions carry two tiers of statistical weight. The synthetic calibration

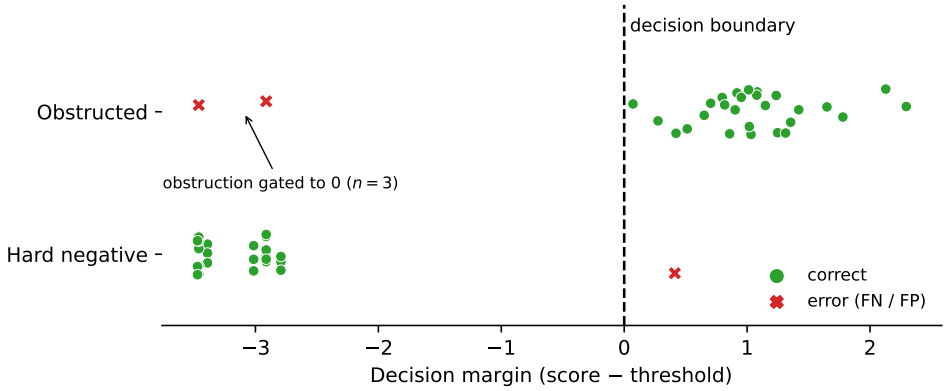


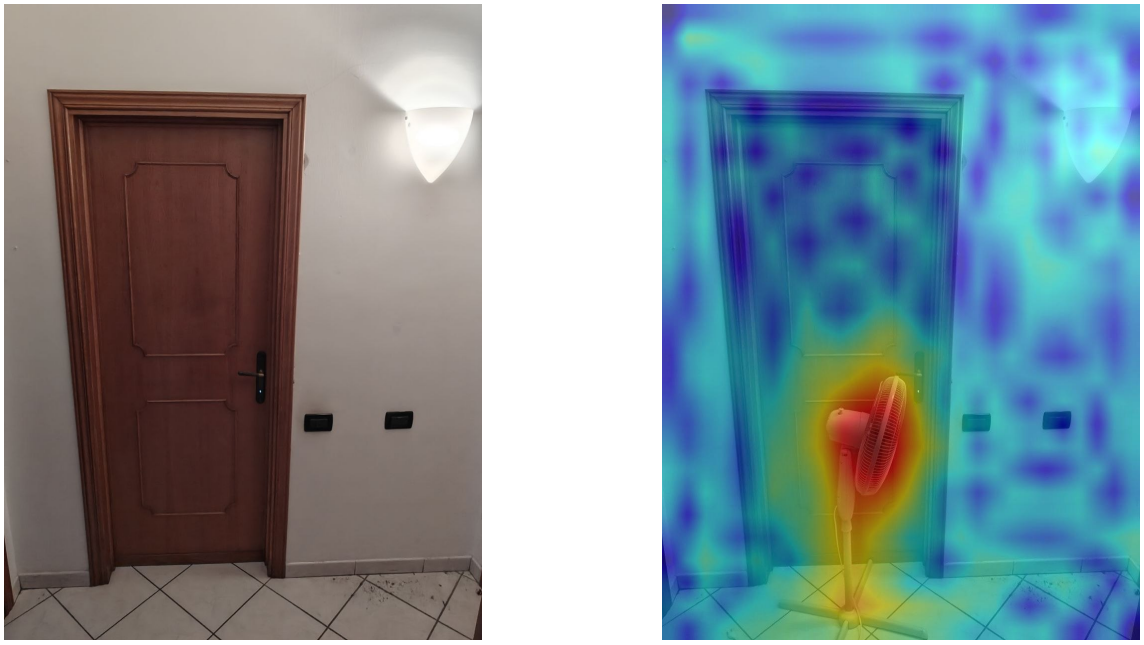
Figure 5.3: Per-sample decision margin (score − per-camera threshold) on D2 at $k = 3$; the decision boundary is the line at 0. The 30 obstructed and 30 hard-negative frames are cleanly separated except for three obstructions gated to a zero score - the gating-limited false negatives, including the thin-stemmed floor fan - and one hard-negative false positive (a glass door whose background changed behind the panel).

phase (Phase 1, $N \approx 1000$) gives quantitative validations with adequate power: the measured deltas of shadow augmentation, decoupling, and pooled σ are stable and generalisable to similar synthetic scenes. The real-world phase (Phase 2, $N \approx$ tens) is a qualitative validation of the complete pipeline: it confirms the expected behaviour of gating plus pooled σ and motivates the k recalibration, but the absolute numbers are illustrative, not a production benchmark. The structural results - augmentation effectiveness, decoupling rationale, pooled-sigma variance reduction, and the discrimination-versus-calibration mechanism - are pipeline properties that generalise; the exact operating-point metrics should be recalibrated for any new deployment using a site-specific σ_{pop} and a k chosen from the application's cost ratio of false negatives to false positives. Larger-scale quantitative validation of the gating on real glass doors, conditioned on a semi-automatically annotated test set, is the natural next step.

5.1.2 Single-Class Obstruction Detection with YOLO (P1)

Evaluation protocol

The evaluation of P1 is organised in two macro-phases (Table 5.8), mirroring the structure used for P3. **Phase 1** tests the *frozen-vocabulary hypothesis*: COCO-pretrained detectors used without any fine-tuning, with a curated mapping of COCO classes onto the obstruction concept. Its role is to establish, at zero training cost, the ceiling of the "no fine-tuning" route and to diagnose *why* it falls short. **Phase 2** fine-tunes the detectors of Table 4.5 on the synthetic corpus and evaluates them at two levels: detection mAP on the generative test set (D3) and image-level operational accuracy on the real on-site set (D2), the latter shared with P3. Unless



(a) Reference (normal) (b) Anomaly heatmap

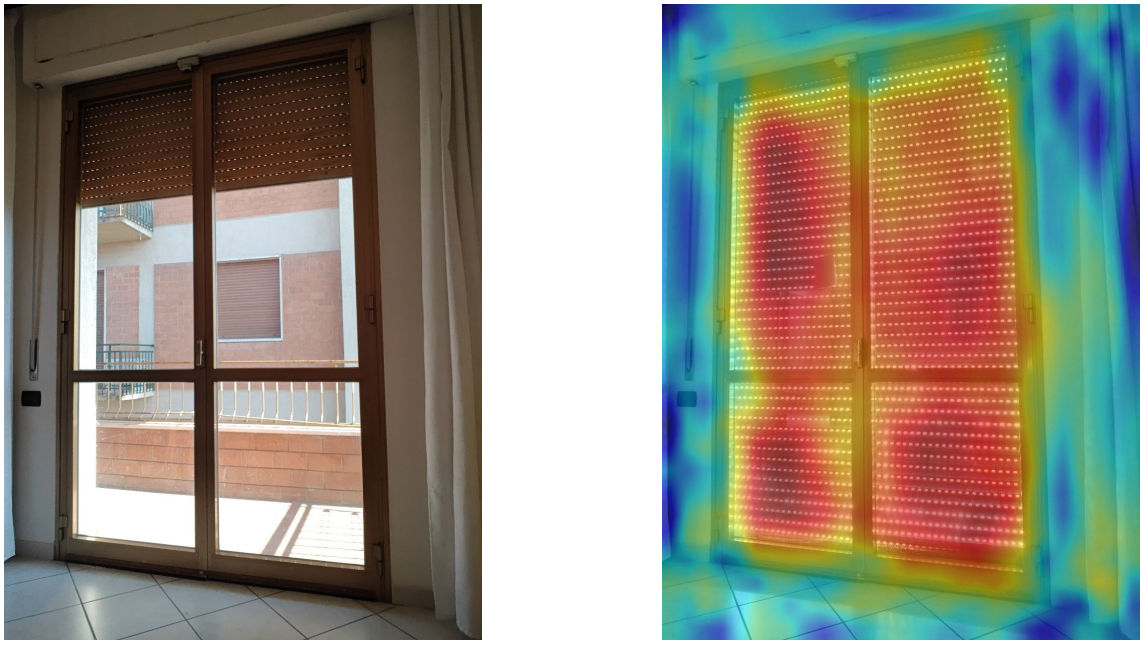
Figure 5.4: Gating-limited false negative (worst case, score 0.000). PatchCore correctly flags the fan head and base as anomalous in the heatmap, but the head rests on the door panel, which is excluded from the frame mask by design. The thin pole - the only part touching the frame and threshold - is too slender to register as anomalous, so the fan does not form a single connected anomaly component bridging the panel to a frame region. The head's component lies entirely in the excluded panel area with no anomalous link reaching the frame, and gating therefore zeros it. The failure is purely geometric, not a detection error.

stated otherwise, image-level decisions use the protocol of Chapter 4 ($\tau_{\text{conf}} = 0.25$, minimum box area 1%).

Phase 1: frozen-vocabulary baselines

COCO class mapping. A COCO-pretrained detector knows 80 classes, none of which is "obstruction". The baseline therefore counts a detection as an obstruction when its class belongs to a hand-curated subset of COCO classes semantically compatible with floor-standing obstacles (initially: bench, backpack, handbag, suitcase, chair, couch, potted plant, bed, dining table, refrigerator). With YOLO11n and this initial mapping, the result on the synthetic corpus is a recall of 0.302 at precision 0.929 - the detector misses about 70% of the obstructions - and transfers to the real set at recall 0.233 with F1 0.304.

A per-class analysis on the 990 positives and 110 clean negatives identifies both the false-positive drivers and the latent signal. The `refrigerator` class fires on 11.8% of the *clean* negatives (against 3.5% of positives): visual inspection of all 49 crops shows it acting as a generic detector of large flat vertical panels - doors,



(a) Reference (normal) (b) Anomaly heatmap

Figure 5.5: Hard-negative false positive (worst case, score 3.873) on a glass door. In the reference the external rolling shutter is up and the glass shows the courtyard outside; in the test frame the shutter is lowered, so the view through the panel changes completely. The background variation is too large for the fixed-camera assumption: the entire glazed area exceeds the anomaly threshold and overlaps the door frame, producing a false detection. This is a calibration limit of the static-background assumption, not a gating failure.

not refrigerators - and it is removed from the mapping. Conversely, objects with wheels are systematically detected under vehicle classes: `bicycle` fires on 9.9% of positives and on *zero* negatives, `motorcycle` on 3.8%/0%, because the detector recognises the wheels of carts and wheelchairs; cylindrical waste bins fire on `toilet` (47 of 48 detections on positives). The refined mapping (removing `refrigerator` and `handbag`, adding `bicycle`, `motorcycle`, `toilet`, `vase`) improves the synthetic metrics substantially (Table 5.9).

The improvement, however, does *not* transfer: on the real test set the refined mapping produces a result identical to the previous one (recall 0.200, F1 0.308) - the `toilet` class, which detected the synthetic cylindrical bins almost perfectly, fires on none of the real rectangular bins. This is a concrete instance of the synthetic-to-real gap: a mapping adjustment validated on clean synthetic crops is useless on real scenes, a warning that recurs throughout this chapter.

Open-vocabulary variant. YOLO-World [6] replaces the fixed vocabulary with text prompts, so the five obstacle categories (plus synonyms) can be requested directly. On the synthetic corpus it is nearly perfect (F1 0.958 at confidence 0.10, FPR

Phase	Data	Focus	Key metrics
1	D1 + D2	Frozen COCO detectors, class mapping, open-vocabulary variant	Image-level recall, F1, FPR
2	D3 + D2	Fine-tuned detectors: localisation quality, real-world accuracy, capacity and architecture axes	mAP, instance/scene recall, F1

Table 5.8: The two macro-phases of the P1 evaluation.

Mapping (YOLO11n, D1)	Recall	Precision	F1	FPR
Initial (10 classes)	0.302	0.929	0.456	0.209
Refined (final)	0.406	0.976	**0.574**	0.091

Table 5.9: Phase 1: effect of the mapping refinement on the synthetic corpus.

0.064); on the real set it reaches recall 0.400 and F1 0.522 - the best frozen result, but still missing 60% of real obstructions. The missed frames are almost all of one kind: a free-standing floor fan blocking two of the six doors, an object outside the prompt vocabulary. Adding explicit prompts for it (*ventilator, medical equipment, machine*) recovers almost nothing (4 weak detections out of 18 frames), showing the object is intrinsically hard for the open-vocabulary embedding, not merely un-prompted.

Frozen ceiling. Table 5.10 collects all frozen baselines on the real set. No frozen model exceeds F1 0.52; the differences among them reflect mostly how liberally each fires generic COCO classes, not their learning capacity. The conclusion of Phase 1 is structural: the obstruction vocabulary of this domain (wheelchair, cart, stretcher, bin, box, fan) is essentially absent from COCO, so no mapping or prompt engineering can close the gap - the detector must be fine-tuned.

Phase 2: fine-tuning

Real-world accuracy (D2). The fine-tuned YOLO11n, trained on the full synthetic corpus, doubles the recall of its frozen counterpart on the real set (0.200 → 0.400) at F1 0.490 (Table 5.11). The raw recall, however, understates what the fine-tuning achieved, because the false negatives are not uniformly distributed: of the 18 missed obstructed frames, 16 contain the same free-standing floor fan already identified in Phase 1 - an object belonging to *none* of the five training categories, hence pure out-of-distribution content - and only 2 are genuine misses of an in-training category (a waste bin, missed in two frames of one door). Restricted to the frames

Model	Type	Recall	Precision	F1	FPR	Param.
YOLO-World-S	open-vocab. transformer	0.400	0.750	**0.520**	0.111	13.4M
Faster R-CNN MobileNet	CNN two-stage	0.400	0.545	0.462	0.278	∼19M
RT-DETR-L	transformer	0.367	0.440	0.400	0.389	33M
Faster R-CNN ResNet50	CNN two-stage	0.267	0.667	0.381	0.111	∼41M
YOLO11n	CNN one-stage	0.200	0.667	0.308	0.083	2.6M
YOLO26n	CNN one-stage	0.167	1.000	0.286	0.000	2.5M

Table 5.10: Phase 1: all frozen baselines on the real test set (D2), image-level, $\tau_{\text{conf}} = 0.25$, refined mapping (YOLO-World uses direct prompts instead). No frozen model reaches operational accuracy.

whose obstacle belongs to a trained category, recall is $10/12 = 0.833$. The fan is the same physical object that defeats P3's spatial gating (Section 5.1), but through an entirely different mechanism: P3 detects it and loses it to a geometric rule, P1 simply has never seen one. The comparison with Phase 1 is nonetheless sobering: the raw F1 of the fine-tuned nano (0.490) does not clearly beat the best frozen baseline (YOLO-World, 0.520), and the advantage emerges only once the out-of-vocabulary frames are accounted for - coverage of the object vocabulary, not model quality, is the binding constraint on this test set.

YOLO11n on D2	TP	FP	TN	FN	Recall	F1
Frozen (refined mapping)	6	3	33	24	0.200	0.308
Fine-tuned	12	7	29	18	0.400	0.490
Fine-tuned, trained categories only	10	–	–	2	0.833	–

Table 5.11: Phase 2 on the real test set (30 obstructed, 36 clear frames): fine-tuning versus the frozen baseline, and the recall restricted to frames whose obstacle belongs to a training category (16 of the 18 false negatives contain the out-of-vocabulary floor fan).

Detection quality (D3). On the generative test set the fine-tuned YOLO11n reaches mAP@50 = 0.866 and mAP@50–95 = 0.703, with precision 0.963 and recall 0.831 at the F1-optimal confidence. The gap between the two mAP levels indicates that the residual weakness is localisation tightness rather than object finding - a reading the next analysis makes precise. (An earlier run of this evaluation forced $\tau_{\text{conf}} = 0.25$ during mAP computation and underestimated mAP@50 by about six points; all reported values follow the correct protocol of Chapter 4.)

Instance-level versus scene-level reading

Qualitative inspection of the D3 errors shows a characteristic failure mode: stacked cardboard boxes are detected as *one* box spanning the pile instead of N separate

instances. This section quantifies that observation and, more importantly, examines what it does - and does not - imply for the task. The analysis was run on the first release of D3 (94 images, 187 instances, hand-assigned category tags).

Each ground-truth instance is classified by matching predictions at IoU ≥ 0.5 (the mAP@50 rule) and, for the misses, by inspecting the best-overlapping prediction: a miss is *fused* when that prediction overlaps the instance but was already claimed by a different ground-truth object (one predicted box spanning several objects), *loose* when a prediction overlaps it without being matched to anything (imprecise localisation), and *isolated* when no prediction lies anywhere near. Of 79 misses, 16 (20%) are fused, 41 (52%) loose, and 22 (28%) isolated - so nearly three quarters of the "missed" instances have a prediction in their vicinity. The per-category breakdown is stark: the `box` category has instance recall 0.281 (27/96) against 0.83–0.96 for the other four categories, and *all* fused misses belong to it. The root cause is in the training data: the copy-paste generator enforces a maximum pairwise IoU of 0.10 between pasted objects (Table 3.2), so the configuration "several heavily overlapping same-class objects" is absent from training by construction, and boxes are the only category the generative model habitually stacks.

Two observations prevent over-correcting for this. First, for heavily occluded stacked objects the ground truth itself is ambiguous: whether three stacked boxes "are" one object or three is an annotation convention (amodal versus modal boxes), and either convention penalises one of the two plausible model behaviours - the instance-level metric is partly ill-posed on such scenes, independently of the model. Second, at the level that matters operationally - is the route obstructed? - the failure disappears entirely: every scene of the first release containing at least one box is flagged (30/30), a higher scene rate than for scenes without boxes (60/64). The scene-level metric, however, is biased in the opposite direction (a scene with five objects counts as detected if one is found), so neither number alone is honest: the two bracket the truth, and the clean signal in between is the *isolated* miss count - 22 instances (12 of them boxes) that no prediction approaches, the only misses that are genuine blind spots under any annotation convention. For a single-class presence task, the stacked-box fusion is thus a metric artefact rather than a functional defect, and no NMS or data intervention targeting it was pursued.

Background-leakage control

As anticipated in Section 3.3.4, every D3 background also appears in the training pool of the copy-paste corpus (107 stems as composite backgrounds, 13 as clean negatives): D3 could reward background memorisation rather than object detection. The control is direct: YOLO11n was retrained on the purged training set (883 positives, 97 negatives) and compared, on the identical D3 release, with the model trained on the full corpus (Table 5.12).

The two models are statistically indistinguishable - the purged model is even marginally better. A complementary check on the first release found no recall advantage on backgrounds seen as training positives over backgrounds seen only as

YOLO11n training set	mAP@50	mAP@50–95	Precision	Recall
Full corpus (990/110)	0.863	0.709	0.951	0.789
Purged (883/97)	0.866	0.703	0.963	0.831

Table 5.12: Background-leakage control on D3: removing every training image that shares a background with the test set leaves the results unchanged.

clean negatives (0.588 versus 0.545). Background overlap therefore does *not* inflate the D3 numbers: the detector is reading the objects, not recognising the scenes. All subsequent experiments nevertheless use the purged training set, as the more defensible protocol.

Capacity and architecture study

With the protocol fixed, the remaining question is whether the choice of detector matters: does more capacity, or a different architecture, buy real generalisation? Seven models - three sizes per YOLO generation plus RT-DETR-L - were trained identically on the purged corpus and evaluated on D3 (Table 5.13).

Model	Param.	Synth. mAP@50–95	Generative test set (D3)			
		(internal val)	mAP@50	mAP@50–95	Prec.	Recall
YOLO11n	2.6M	0.972	**0.866**	0.703	0.963	0.831
YOLO11s	9.4M	0.986	0.867	0.709	0.923	0.837
YOLO11m	20.1M	0.984	0.856	0.708	0.965	0.837
YOLO26n	2.5M	0.982	0.852	0.708	0.969	0.789
YOLO26s	9.9M	0.978	0.854	0.698	0.952	0.840
YOLO26m	21.8M	0.982	0.855	**0.716**	0.952	0.833
RT-DETR-L	32.8M	**0.991**	0.847	0.702	0.943	0.801

Table 5.13: Capacity and architecture study, all models trained identically on the purged corpus. A 12.6× parameter range moves D3 mAP@50 by two points with no monotone trend, while the internal synthetic validation is saturated for every model.

Three findings emerge. First, *capacity buys nothing*: across a 12.6× parameter range, D3 mAP@50 spans 0.847–0.867 with no monotone trend - the spread is fold-to-fold noise, not signal. This echoes, in a more extreme form, the small nano-to-XLarge gain (+3.5 mAP points) reported by Miglionico et al. on real PPE data [20]: on a synthetic training corpus the capacity axis saturates entirely. Second, *the internal validation is saturated and non-predictive*: every model scores 0.97–0.99 on the held-out synthetic fold, and the ranking there does not correlate with D3 - RT-DETR-L is the best model on synthetic data and the worst on D3. Model selection on the synthetic validation would pick the wrong model; only an external test set can rank these detectors. Third, the bottleneck is the data, not the model: since no architecture or size moves the external metric, the residual gap to perfect

detection lies in the synthetic-to-real distribution shift and in vocabulary coverage - consistent with the fan analysis on D2.

RT-DETR training instability. The RT-DETR result required one protocol repair worth documenting. Trained with the default automatic mixed precision, the run collapsed: validation losses turned `NaN` sporadically from epoch 4 and permanently from epoch 15, at which point every metric dropped to zero and early stopping ended the run at epoch 34; the usable checkpoint was the pre-collapse epoch 14. This is a documented failure mode of RT-DETR under AMP - fp16 overflow in the bipartite-matching cost matrix [34] - and the fix prescribed by the framework documentation, disabling AMP with all else unchanged, resolved it completely: the retrained run completed all 150 epochs with zero `NaN` and improved on D3 from mAP@50–95 = 0.645 to 0.702 and recall from 0.741 to 0.801. The single-variable repair attributes the instability to AMP with certainty; Table 5.13 reports the repaired run. Even so repaired, the transformer does not surpass the nano CNNs on D3 while costing 12× their memory - reinforcing, now on equal training footing, that it is not the right candidate for this deployment.

Discussion

Across both phases, the failure modes of the detector pipeline are diagnosable and consistent. Phase 1 establishes that the frozen-vocabulary route fails for a structural reason - the domain's obstacle categories are absent from COCO - and that the failure survives both mapping refinement and open-vocabulary prompting; the per-class analysis (wheels detected as bicycles, bins as toilets) shows the detectors *see* the objects but cannot name them, which is precisely the deficit fine-tuning repairs. Phase 2 shows that fine-tuning on synthesis-derived labels transfers to real scenes for the categories it covers (recall 0.833 on trained categories) and fails completely outside them (the floor fan, 16 of 18 false negatives): the binding constraint is vocabulary coverage, not model choice, since a 12.6× capacity range and a CNN-to-transformer architecture change leave the external metrics unmoved. The practical selection criterion then reduces to cost, and the smallest model - YOLO11n, 5 MB - is the rational deployment candidate, a conclusion reached by measurement rather than by assumption.

The metric analysis of Section 5.1.2 carries a lesson that extends beyond P1: for a single-class presence task, instance-level detection metrics and scene-level ones bracket the operational truth from opposite sides, and on scenes with heavily occluded same-class objects the instance ground truth is itself convention-dependent. Reporting a single headline number would mislead in either direction; the defensible statement is the decomposition - which misses are localisation artefacts and which are genuine blind spots.

As for P3, the conclusions carry two tiers of statistical weight. The synthetic and generative evaluations (D1, D3; hundreds of images) support the structural findings - the vocabulary bottleneck, the capacity plateau, the saturation of the internal

validation. The real-set numbers (D2; tens of frames) are a qualitative validation: they confirm the expected behaviour and localise the failure modes, but the absolute values are illustrative. The natural next step, enabled by the fold infrastructure already in place, is a cross-validated estimate of the fold-to-fold variance of the D3 metrics, which would put formal uncertainty bounds under the model comparison of Table 5.13.

5.2 Application Results

[Analysis of the end-to-end application results - to be completed.]

Chapter 6

Conclusions

[Summary and future work]

Appendix A

Questa è un'appendice

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Questa è un'altra sezione, ma non viene inserita nell'indice

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Bibliography

[1] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9592–9600, 2019.

[2] Roel Bouman and Tom Heskes. Autoencoders for anomaly detection are unreliable, 2025.

[3] Yunkang Cao, Yuqi Cheng, Xiaohao Xu, Yiheng Zhang, Yihan Sun, Yuxiang Tan, Yuxin Zhang, Xiaonan Huang, and Weiming Shen. Visual anomaly detection under complex view-illumination interplay: A large-scale benchmark. arXiv preprint arXiv:2505.10996, 2025.

[4] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, pages 213–229, 2020.

[5] Dayou Chen, Long Chen, Yi Yang, Qiuchen Lu, Craig Hancock, Russell Lock, and Simon Sølvsten. A knowledge-driven vision-language reasoning approach for automated fire safety compliance checking in operational buildings. SSRN preprint, 2025. https://ssrn.com/abstract=5387267.

[6] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16901–16911, 2024.

[7] Niv Cohen and Yedid Hoshen. Sub-image anomaly detection with deep pyramid correspondences. arXiv preprint arXiv:2005.02357, 2020.

[8] Thomas Defard, Aleksandr Setkov, Angélique Loesch, and Romaric Audigier. PaDiM: a patch distribution modeling framework for anomaly detection and localization. In *Pattern Recognition. ICPR International Workshops and Challenges*, pages 475–489, 2021.

[9] Huu-Thanh Duong, Viet-Tuan Le, and Vinh Truong Hoang. Deep learning-based anomaly detection in video surveillance: A survey. *Sensors*, 23(11):5024, 2023.

[10] M. Fikret Ercan and Asha Mathyalakan. Computer vision based safety inspection in public housing. In *TENCON 2024 – IEEE Region 10 Conference*, pages 1398–1401, 2024.

[11] Chengjian Feng, Yujie Zhong, Yu Gao, Matthew R. Scott, and Weilin Huang. TOOD: Task-aligned one-stage object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3510–3519, 2021.

[12] Gionatan Gallo, Francesco Di Rienzo, Federico Garzelli, Pietro Ducange, and Carlo Vallati. A smart system for personal protective equipment detection in industrial environments based on deep learning at the edge. *IEEE Access*, 10:110862–110878, 2022.

[13] Google DeepMind. Gemini 2.5 flash image. https://deepmind.google/models/gemini/, 2025. Generative image editing model, `gemini-2.5-flash-image`.

[14] Guillermo Iglesias, Edgar Talavera, and Alberto Díaz-Álvarez. A survey on GANs for computer vision: Recent research, analysis and taxonomy. *Computer Science Review*, 48:100553, 2023.

[15] Glenn Jocher and Jing Qiu. Ultralytics YOLO11. https://docs.ultralytics.com/models/yolo11/, 2024. Ultralytics, version 11.0.0.

[16] Shengtai Ju and Amy R. Reibman. Shadow augmentation for handwashing action recognition: from synthetic to real datasets. arXiv preprint arXiv:2410.03984, 2024.

[17] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[18] Yuxuan Lin, Yang Chang, Xuan Tong, Jiawen Yu, Antonio Liotta, Guofan Huang, Wei Song, Deyu Zeng, Zongze Wu, Yan Wang, and Wenqiang Zhang. A survey on RGB, 3D, and multimodal approaches for unsupervised industrial image anomaly detection. arXiv preprint arXiv:2410.21982, 2024.

[19] Wenyu Lv, Yian Zhao, Qinyao Chang, Kui Huang, Guanzhong Wang, and Yi Liu. DETRs beat YOLOs on real-time object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16965–16974, 2024.

[20] Giustino Claudio Miglionico, Francesco Di Rienzo, Pietro Ducange, Francesco Marcelloni, and Carlo Vallati. A comparative analysis of models for real-time personal protective equipment detection on edge devices. In *2025 International Joint Conference on Neural Networks (IJCNN)*, 2025.

[21] Giustino Claudio Miglionico, Pietro Ducange, Francesco Marcelloni, Carlo Vallati, and Francesco di Rienzo. Performance evaluation of YOLOv5 on edge devices for personal protective equipment detection. In *2024 International Joint Conference on Neural Networks (IJCNN)*, 2024.

[22] Payal Mittal. A comprehensive survey of deep learning-based lightweight object detection models for edge devices. *Artificial Intelligence Review*, 57:242, 2024.

[23] Zainab Ouardirhi, Sidi Ahmed Mahmoudi, and Mostapha Zbakh. Enhancing object detection in smart video surveillance: A survey of occlusion-handling approaches. *Electronics*, 13(3):541, 2024.

[24] Devashree R. Patrikar and Mayur Rajaram Parate. Anomaly detection using edge computing in video surveillance system: review. *International Journal of Multimedia Information Retrieval*, 11:85–110, 2022.

[25] Xuebin Qin, Zichen Zhang, Chenyang Huang, Masood Dehghan, Osmar R. Zaiane, and Martin Jagersand. U^2-net: Going deeper with nested u-structure for salient object detection. *Pattern Recognition*, 106:107404, 2020.

[26] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[27] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[28] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14318–14328, 2022.

[29] Andres Sanin, Conrad Sanderson, and Brian C. Lovell. Shadow detection: A survey and comparative evaluation of recent methods. *Pattern Recognition*, 45(4):1684–1695, 2012.

[30] Marcos C. S. Santana, Leandro Aparecido Passos, Thierry P. Moreira, Danilo Colombo, Victor Hugo C. de Albuquerque, and João Paulo Papa. A novel siamese-based approach for scene change detection with applications to obstructed routes in hazardous environments. *IEEE Intelligent Systems*, 35(1):44–53, 2020.

[31] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations (ICLR)*, 2018.

[32] Hyo-Ju Shin, Sang-Pil Jung, and Jin-Wook Kim. Improving object classification accuracy for detecting obstacles on indoor evacuation paths using YOLOv8. *KIEAE Journal*, 25(6):5–14, 2025.

[33] Hamed Talebi, Nahid Khoshk Angabini, Arezoo Sarkheyli-Hägele, and Erdal Akin. Real time exit blockage detection via door-centric change monitoring for emergency evacuation. In *Proceedings of the 18th International Conference on Agents and Artificial Intelligence (ICAART)*, volume 3, pages 2648–2657, 2026.

[34] Ultralytics. Reference for `ultralytics/models/rtdetr/train.py`. https://docs.ultralytics.com/reference/models/rtdetr/train/, 2025. Accessed July 2026. The `RTDETRTrainer` docstring warns: "AMP training can lead to NaN outputs and may produce errors during bipartite graph matching".

[35] Ultralytics. YOLO26: Unified real-time end-to-end vision models. https://docs.ultralytics.com/models/yolo26/, 2025.

[36] Wikipedia contributors. Johnson–lindenstrauss lemma — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Johnson%E2%80%93Lindenstrauss_lemma&oldid=1357719950, 2026. [Online; accessed 10-June-2026].

[37] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2016.

[38] Syed Farhan Alam Zaidi, Jaehun Yang, Muhammad Sibtain Abbas, Rahat Hussain, Doyeop Lee, and Chansik Park. Vision-based construction safety monitoring utilizing temporal analysis to reduce false alarms. *Buildings*, 14(6):1878, 2024.